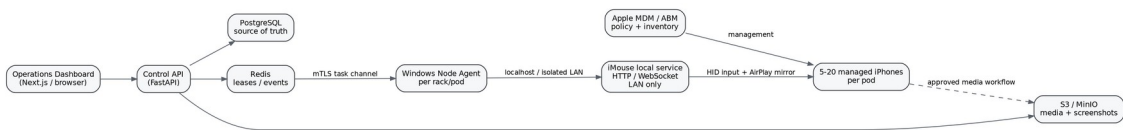


PHONE FARM

MASTER TECHNICAL HANDOFF

Claude-ready architecture, evidence ledger, experiments, implementation specification and engineering reference



XYZ MGMT - Confidential / Internal Technical Review

Version 2.0 - 26 August 2026

This master PDF is designed to be handed directly to an engineering model or engineer. Part A resolves the new internal research into decisions and build gates. Part B preserves the earlier 91-page engineering reference guide for deep background.

READ THIS FIRST - instructions to Claude / technical lead

Primary mission

Convert the evidence in this document into a measured, maintainable device-fleet system. Do not begin by writing automation scripts. Begin by closing the blocking unknowns, inventorying what XYZ already owns, and producing Architecture Decision Records (ADRs).

The internal brief explicitly says that the project has hands-on knowledge that is not written down. Dim has tested a Chinese mirroring/control product, handled devices and hardware, and knows which attempts failed. Treat his direct observations as first-party experimental data. However, distinguish observation from interpretation: first-party OS or platform documentation takes precedence for claims about operating-system semantics, platform policy, and documented enforcement behavior.

Non-negotiable execution rules

- **1. Evidence before code.** Every decision that could change the hardware, host OS, networking model or control stack must be linked to a source, a measured test, or an explicitly labeled assumption.
- **2. Ask Dim before buying.** Obtain the exact name/version of the control product he tested, the models and state of phones already owned, the 20-device enclosure details, cabling, power supply ratings, and all failures observed.
- **3. Build the durable pilot first.** The default implementation target in this handoff is a 5-device durable pod, then 10, then 20. Do not jump to a 100-300-device disposable fleet.
- **4. Human-in-the-loop for third-party social UIs.** The control system may expose click/swipe/OCR APIs, but this handoff does not authorize autonomous behavior intended to imitate users, bulk-create accounts, manufacture engagement, evade enforcement, or bypass platform controls.
- **5. No reset/identity assumptions.** A factory reset is a lifecycle operation, not a guaranteed identity reset. Apple DeviceCheck and Google Play Integrity device recall demonstrate reset-surviving anti-abuse state mechanisms.
- **6. No “undetectable” claims.** Hardware input, network egress, device state, behavior and content can all produce observable signals. If a claim depends on invisibility, rewrite it as a testable hypothesis.
- **7. Keep a decision ledger.** Every resolved unknown becomes an ADR with evidence, tradeoffs, chosen option, rejected alternatives and rollback path.
- **8. Keep the source brief intact.** Do not silently rewrite operator testimony into fact. Preserve operator claims as claims and attach confidence labels.

Definition of done for the first engineering cycle

- A documented inventory of every device, chassis, control board, cable, power supply and software license already in hand.
- A two-device evidence bench that answers the iMouse input-type question, network-egress question and baseline stability questions.
- A selected control product with a security review and a documented reason for choosing it.
- A 5-device pod operating for at least one continuous test window with telemetry, media import, human remote control and recoverable node-agent failures.
- A functioning control-plane repository with PostgreSQL-backed inventory, task leasing, audit events, operator dashboard and safe device actions.
- A measured power/thermal/network capacity sheet that replaces vendor assumptions with local observations.
- A written go/no-go decision before any 20+ device expansion.

What this handoff intentionally will not build

Excluded implementation scope

Do not implement fingerprint spoofing, location falsification, bulk account registration, automated friend/follow/add pipelines, scraped-user targeting, fake engagement, CAPTCHA/rate-limit bypass, automatic ban replacement, or any workflow whose design objective is to conceal coordinated behavior from a platform. The research may mention such operator practices because they explain the architecture; they are not implementation requirements.

1. Document map and authority model

This PDF has two layers. Part A (this handoff) is the current decision layer and supersedes earlier assumptions where it says so. Part B is the original 91-page engineering reference covering power, racks, networking, proxies, MDM, Android Enterprise, control planes, analytics, security, incident response and scaling. Use Part B for depth; use Part A for current decisions and unresolved questions.

1.1 Evidence hierarchy

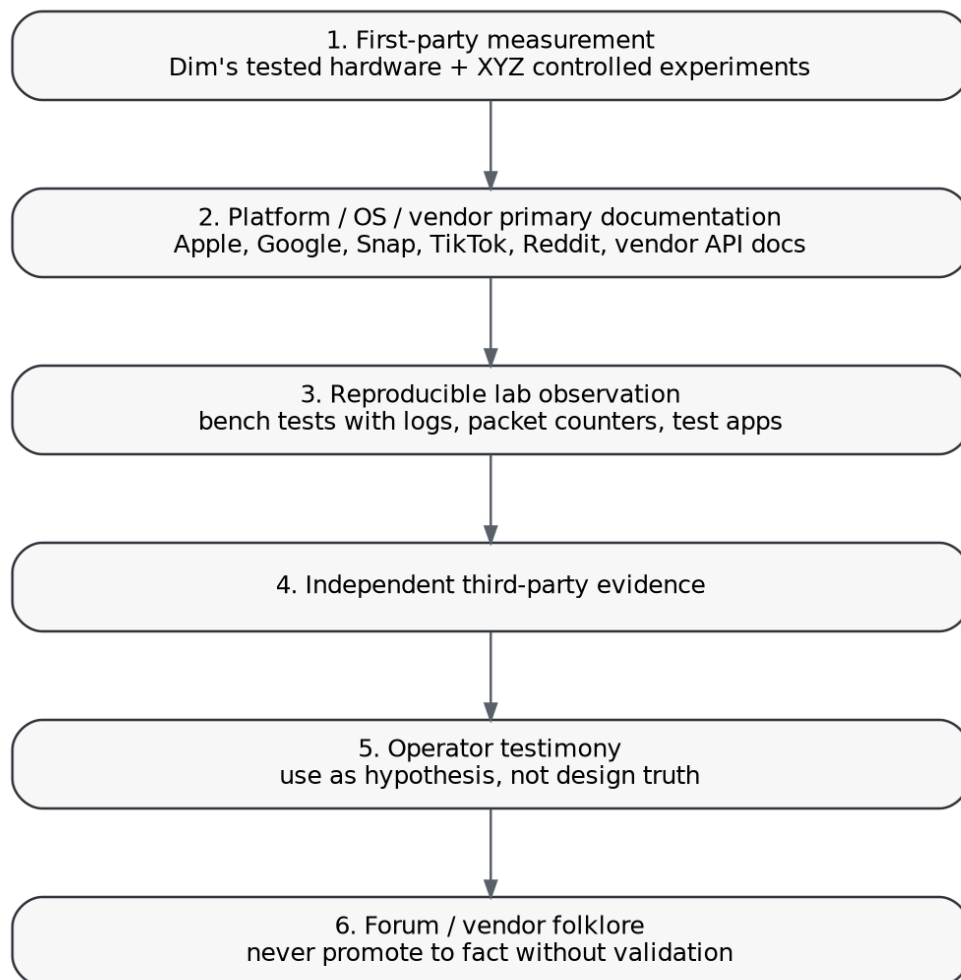


Figure A1 - Evidence hierarchy used in this master handoff.

Rank	Evidence class	How Claude should use it
1	XYZ first-party measurement / Dim observation	Treat the observed event as real for the tested version/configuration. Preserve logs and exact conditions.

Rank	Evidence class	How Claude should use it
2	Apple / Google / Snap / TikTok / Reddit primary documentation	Use for OS semantics, supported management features, published policy and documented enforcement.
3	Reproducible controlled bench test	Promote to project fact only with test ID, date, hardware/software versions and evidence artifacts.
4	Vendor primary documentation	Use for product capabilities and requirements; still verify security, stability and actual behavior.
5	Operator testimony	Use to form hypotheses and business scenarios. Do not convert yield, ban cycle or detection theories into facts.
6	Forum/vendor folklore	Record only if useful for a test; never make an architectural commitment from it.

1.2 Confidence vocabulary

Label	Meaning in this document
VERIFIED	Supported by current first-party documentation or XYZ controlled measurement.
VERIFIED - LIMITED	Mechanism is documented, but its use by a specific platform is not established.
VENDOR-STATED	Published by the product vendor; requires local validation.
HYPOTHESIS	Plausible and testable but unresolved.
CONTRADICTED / SUPERSEDED	New evidence directly weakens or replaces the earlier claim.
MODELLED	Engineering or economic calculation based on stated assumptions, not observed production data.
POLICY-BLOCKED	Technically possible but outside this implementation scope because it conflicts with platform rules or the project safety boundary.

1.3 Source authority nuance: “Dim wins”

The internal brief says “if this brief and Dim disagree, Dim wins on anything he has personally tested.” Preserve the intent, but scope it precisely. If Dim reports that software version X mirrored 12 phones for four hours, that observation outranks desk research about whether the vendor product works. If Dim concludes that a platform cannot detect a hardware input method, that conclusion does not outrank Apple’s event-model documentation. Separate “what happened” from “why it happened.”

1.4 Provenance register

ID	Source	Role
I01	Phone Farm - Technical Brief v1.1, 26 Aug 2026	Internal operator research, internal n=1 measurement, open questions, economics.
I02	OFM Phone Farm Engineering Guide, 26 Aug 2026	Existing 91-page infrastructure/control-plane reference; included as Part B.
Pxx	Platform/OS first-party sources listed in the verification ledger	Correctness anchor for OS behavior, support floors, management and policy.
Vxx	Vendor documentation (notably iMouse/Some3C)	Product capability/requirements; security and stability still require validation.
Txx	XYZ controlled test artifacts	Highest-value project evidence once executed.

2. Executive architecture decision

Recommended baseline

Build Option 2 first: a small durable set of organization-owned, current-supported phones, instrumented as a managed fleet. Use official APIs/schedulers where available, a local device console for tasks that genuinely require a phone, and human approval for third-party UI actions. Treat 20 devices as one pod and scale by replicating stable pods, not by making one giant controller.

2.1 Why the 100-300 disposable model is not the engineering baseline

The high-volume Snapchat model described by Operator A is operationally defined by repeated account/device replacement after enforcement. Snap currently documents device bans (SS06/SS18), an SS07 ban when too many accounts are associated with a device, and states that banned devices cannot create new accounts. Snap's March 2026 sexual-content policy also prohibits offers or solicitation of online sexual subscription services. Those facts make "replace the batch after bans" a poor foundation for a maintainable production system, regardless of any reported traffic yield.

Sources: [P01-P03]

The user's internal business observation remains useful: Snapchat can generate measurable traffic at $n=1$, and Operator A's claims may be worth studying as market intelligence. But Claude should not turn enforcement cycling into an automated workflow. If leadership wants further evidence, use controlled experiments with authorized accounts and policy-compliant content, then measure traffic, workload, content demand and account health without building ban replacement or scraped-user acquisition pipelines.

2.2 Option comparison after verification

Dimension	Option 1 - disposable high-volume	Option 2 - durable managed set	Master decision
Scale	100-300 devices	5-30 devices	Start 5 -> 10 -> 20
Core optimization	Provision/replacement throughput	Reliability, quality, durable assets	Durability
Automation pressure	Very high	Low to moderate	Human-in-loop
Policy risk	High: repeated enforcement is part of model	Lower if authorized and compliant	Prefer Option 2
Hardware strategy	Cheapest compatible handsets	Supported, serviceable devices	Current OS support first
Control system	Would require large-scale scripting	Remote console + internal automation	Internal automation only
Data quality	Operator claims dominate	Can create clean first-party metrics	Build evidence
Operational readiness	Internal brief already questions readiness	Matches current team size better	Proceed as pilot

2.3 Minimum recommended iPhone floor

As of the research cut, Apple lists iPhone 11 and iPhone SE (2nd generation) among devices compatible with iOS 26.5. By contrast, the current Instagram App Store listing requires iOS 16.3 or later, Snapchat requires iOS 15.0 or later, and TikTok requires iOS 14.0 or later. An iPhone 6s can run the current Snapchat floor but cannot run iOS 16, so it cannot satisfy the current Instagram requirement. For a durable multi-platform set, the acquisition floor should therefore be a model on Apple's current supported-OS list, not merely a phone that launches one app today.

Sources: [P13-P17]

Procurement rule

Do not purchase by app minimum alone. Require: current security-update eligibility, healthy battery, intact SIM/camera if needed, known MDM/supervision path, compatible control connector, and at least one expected major-OS runway. Re-check app minimum versions on purchase day.

3. Verification ledger - what the new research changes

3.1 Snapchat device bans: verified; mechanism is not

Snap explicitly documents SS06/SS18 as device bans tied to abuse or repeated violations and SS07 as a device ban caused by too many accounts being associated with a device. Snap also says a banned device cannot create new accounts and that used/refurbished buyers can inherit a device ban. Therefore, “device-level enforcement exists” is VERIFIED.

Apple DeviceCheck separately proves that an iOS developer can store two bits of per-device server state that persist across app reinstall, transfer between users and even Erase All Contents and Settings. This demonstrates a plausible technical primitive for reset-surviving state, but there is no evidence in the reviewed sources that Snapchat specifically uses DeviceCheck for its device-ban implementation. The correct statement is therefore: persistent device enforcement is observed and documented by Snap; DeviceCheck is one documented iOS capability that can support persistent per-device state, not a proven explanation of Snapchat’s implementation.

Sources: [P01-P05]

3.2 Android reset recovery: earlier confidence must be reduced

Android’s base identifier guidance still favors resettable identifiers and restricts access to non-resettable hardware identifiers such as IMEI, serial and MAC for ordinary third-party apps. That supports the brief’s criticism of simplistic “the app just reads IMEI/MAC” vendor explanations. However, Google’s 2026 Play Integrity device recall beta can store three developer-defined values for a device on Google servers and recall them after reinstall or device reset. Google explicitly lists repeat abuse and repeated abusive account creation as use cases.

Consequence: “Android has no reset-surviving anti-abuse primitive” is SUPERSEDED. A wipe may clear local identifiers and application data, but it is not a dependable economic recovery strategy when platform-integrity services can maintain backend state.

Sources: [P06-P07]

3.3 Hardware pointer input: the OS signal is real; iMouse-specific behavior is unresolved

Apple documents `UIApplicationSupportsIndirectInputEvents`. If enabled, clicks from an indirect pointing device produce `UITouch.TouchType.indirectPointer`; if disabled they are delivered as direct touches for compatibility. If the key is absent, iOS 17 and later defaults to indirect-input support, while versions before iOS 17 default to the compatibility behavior. Apple’s iOS 17 release notes also describe the SDK-dependent default change.

This validates the internal brief’s core concern that pointer input can be distinguishable at the OS/API level. It does not prove that the iMouse OTG board is delivered to Snapchat/Instagram/TikTok as `indirectPointer`, because the actual AssistiveTouch/Full Keyboard Access/HID path must be measured. Do not freeze OS versions to hide this signal. Instead, build a tiny owned test app that logs `UITouch.type` and compare finger, Bluetooth mouse and iMouse input across available iOS versions.

Sources: [P08-P10]

3.4 iMouse product reality: verified vendor product, but security posture requires containment

Some3C/phones.farm publicly describes iMouse as a one-board-per-phone iOS control system requiring no jailbreak, signing or on-device app, with HTTP/WebSocket interfaces, OCR/image/color functions and a Windows-only host. Vendor setup documentation says the PC and iPhone must be on the same network and recommends trusting/excluding or even uninstalling endpoint security if it interferes. The API documentation exposes local HTTP examples and an allow-list concept.

Conclusion: the architecture described by Operator B is credible at the product-capability level. But the product must be treated as a high-trust local control appliance, not as a safe Internet-facing service. Never expose its API or

AirPlay ports to the public Internet. Run it on a dedicated Windows node in an isolated management zone; have the XYZ node agent call the vendor API locally; expose only the XYZ-authenticated control channel outward.

Sources: [V01-V05]

3.5 Same-LAN mirroring creates a real egress design question

The vendor requires the phone and control PC to share a network segment for mirroring. The internal brief correctly identifies the resulting ambiguity: when Wi-Fi is associated and cellular is active, which path carries the third-party app's Internet traffic? A public-IP check in Safari is useful but insufficient to prove every app's route. The correct test combines a controlled local-only Wi-Fi network, router counters/packet logs, cellular usage counters and a neutral endpoint you own.

Do not infer per-device carrier egress

A phone having a SIM does not prove that its application traffic uses the cellular interface while it is simultaneously associated to the control WLAN. Egress is a measurement, not a configuration label.

3.6 App compatibility changes the hardware decision

The internal brief's iPhone 6s conclusion is directionally correct and can now be anchored to current App Store requirements. Snapchat currently requires iOS 15.0; Instagram requires iOS 16.3; TikTok requires iOS 14.0. Apple's current iOS 26.5 support list begins at iPhone 11 / SE 2. Therefore, a 6s-class acquisition strategy is a single-platform, end-of-life bet, while iPhone 11/SE2-or-newer is the current minimum class for a durable set.

Sources: [P13-P17]

3.7 The five-post observation remains a hypothesis

XYZ's internal n=1 test is valuable because it is first-party: the account moved from roughly 3-4 free subscribers to 19 with 147 link clicks after the fifth public-profile story, then to 29 subscribers and 182 clicks on the following measurement. The coincidence with Operator A's "at least five photos" rule justifies an experiment; it does not establish a platform threshold. Timing, account age, content, network and random distribution effects remain confounders.

Claude should model this as hypothesis H-SNAP-01 and design a small randomized or staggered test with predeclared metrics. Do not encode "5 posts unlock distribution" as business logic unless replicated.

4. Blocking experiments - run these before major coding or procurement

These tests are ordered by decision impact. Each test should create a markdown report under `/docs/tests/` with hardware IDs, OS/software versions, exact procedure, timestamps, raw evidence references, result, confidence and follow-up. Where a test uses a custom app, keep it limited to owned test software and neutral endpoints.

4.1 TEST-IOS-INPUT-001 - identify the actual iMouse touch type

Field	Specification
Question	Does the exact iMouse board/software path produce direct or indirectPointer touch events in an owned UIKit test app?
Devices	At least one iOS 16-era device if already owned and one iOS 17+ / current-supported device. Do not procure obsolete hardware solely for this test.
Inputs	Finger tap; standard pointing device; iMouse board click.
Test app	Minimal UIKit app logging UITouch.type, event timestamp, coordinates and app build SDK; include visible counter for evidence.
Variants	Build once with UIApplicationSupportsIndirectInputEvents omitted, once YES, once NO.

Field	Specification
Evidence	Device screen recording, app log, iOS version, app SDK version, iMouse version, board ID.
Pass condition	Actual event type for each combination is documented and repeatable.
Decision effect	Determines whether iMouse is acceptable as a human remote-control method and what observability caveat must be documented. It is not a basis for evasion.

4.2 TEST-NET-EGRESS-001 - Wi-Fi control plane versus cellular data plane

Field	Specification
Question	Can the phone maintain local mirroring/control on the control WLAN while normal Internet access leaves over cellular, and is that stable?
Topology	Dedicated AP/SSID and controller on the same private subnet. First run with normal Internet gateway; second run with WLAN Internet egress intentionally unavailable while local subnet remains reachable.
Measurement	Neutral HTTPS endpoint owned by XYZ logs source address; router records WAN counters; device records cellular-data delta; iMouse mirror stability is logged.
Scope	Test Safari/owned diagnostic app. Do not infer a third-party app route without independent observation.
Evidence	Endpoint log, router counters, cellular counter screenshots, mirror uptime, route conditions.
Pass condition	A stable, documented combination is found or the requirement is rejected as unreliable.
Decision effect	Selects normal Wi-Fi, cellular, or managed VPN architecture. Do not treat the result as an anti-detection feature.

4.3 TEST-IOS-CONTROL-001 - iMouse stability and security baseline

- Install on a dedicated Windows test host, not a personal workstation.
- Record installer hash, version, services, listening ports, startup behavior and update mechanism.
- Keep endpoint protection enabled. If a narrow exclusion is genuinely required, document exactly which folder/process and why; do not uninstall security software as a default.
- Bind two phones and run mirror/control continuously. Record reconnects, dropped frames, CPU/GPU/RAM/NIC, USB board disconnects and service restarts.
- Confirm whether API endpoints have authentication/TLS. If not, enforce host firewall + management-VLAN ACL and bind/allow-list only required addresses.
- Exercise only neutral actions in the test: screen mirror, screenshot, clipboard test, safe settings navigation, media import to owned test assets.

4.4 TEST-MEDIA-IOS-001 - content delivery to Photos

The vendor API states that photos/videos can be sent to the phone album. Validate the exact mechanism, format constraints, import latency, duplicate behavior, metadata preservation and failure recovery on two current devices. The goal is a reliable cloud -> queue -> local node -> device Photos import workflow. Do not tie this test to an automated social-post action.

4.5 TEST-CAPACITY-001 - devices per Windows node

There is no credible universal “devices per host” constant. The capacity boundary depends on mirroring resolution/fps, codec, CPU/GPU decode, NIC, AirPlay implementation, iMouse process behavior, USB controller topology and operator UI. Benchmark 1, 5, 10 and 20 devices on the actual host. Only test 40 on one host if 20 is comfortably below resource and failure thresholds; otherwise scale by pods.

Metric	Collect at each concurrency level	Initial engineering gate
CPU/GPU	mean, p95, peak; per-process if available	p95 < 70% recommended starting gate
RAM	working set, commit, paging	no sustained paging; >25% headroom
NIC	RX/TX Mbps, drops, errors	<60% sustained link utilization; no drops
Mirror quality	fps, reconnect count, time-to-first-frame	no unexplained reconnect storm
Control latency	click-to-visible-result in owned test UI	p95 target defined from operator usability test
Node recovery	restart iMouse service / agent / host	devices recover without database corruption
USB	board enumeration, disconnects by controller/hub	no silent device loss over soak test

The utilization percentages above are design starting points, not vendor guarantees. Replace them with local measurements after the first soak test.

4.6 TEST-THERMAL-001 - 20-slot thermal and charging characterization

Use the actual chassis. Run empty, 5-device, 10-device and full-load stages. Log ambient intake/exhaust, representative device temperature indications, charging state, fan state and power draw. Apple specifies 0-35 C ambient operation and can slow or pause charging when temperature is high; the rack should target substantially below the upper limit to preserve battery life and headroom.

Sources: [P18-P19]

5. Recommended reference architecture - durable iOS pod

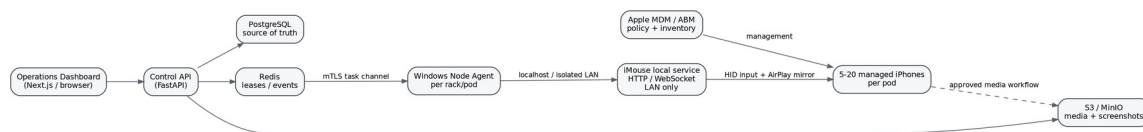


Figure A2 - Recommended control-plane architecture. The vendor controller remains local; XYZ owns the authenticated outer control plane.

5.1 Pod as the scaling unit

Treat one physical pod as the smallest independently recoverable unit. A pod contains 5-20 phones, one dedicated Windows node, one control-board/chassis set, one control WLAN segment, labeled power distribution and a telemetry identity. A failure in one pod should not take down another pod. Scaling from 20 to 40 devices means adding a second pod unless the capacity test proves that a single node remains comfortably within thresholds and the blast radius is acceptable.

5.2 Component responsibilities

Component	Responsibility	Must not do
Operations dashboard	Inventory, health, queues, operator attention, audit search, approval views	Store platform passwords in browser local storage; directly expose vendor API
Control API	AuthN/AuthZ, business rules, task creation, state transitions	Issue unrestricted arbitrary mouse macros to third-party apps
PostgreSQL	Authoritative inventory, assignments, tasks, audit metadata	Store raw secrets in plaintext
Redis	Short-lived task leases, heartbeats, event fan-out	Become source of truth
Object storage	Approved media, screenshots/evidence, logs/artifacts	Serve unapproved content to devices
Windows node agent	Local adapter to iMouse, media staging, health collection	Accept unauthenticated commands from the Internet
iMouse local service	Mirror/control hardware and vendor-supported device operations	Be Internet-exposed or trusted as the global orchestration layer
MDM / ABM	Supervision, enrollment, restrictions, managed apps, inventory/policy	Replace the app/platform's own authorization rules

Component	Responsibility	Must not do
Phone	Human-operated publishing endpoint / platform client where legitimately required	Be treated as disposable identity material

5.3 Task taxonomy

The control plane should implement an allow-list of task types. This prevents a general-purpose device-control API from quietly becoming a platform automation engine.

Task type	Automation level	Evidence requirement
health_check	Automatic	heartbeat + state snapshot
capture_screenshot	Automatic for diagnostics	reason + retention policy
import_media	Automatic after asset approval	asset hash + approval ID + device
sync_inventory	Automatic	MDM/vendor state
restart_local_control_service	Automatic with retry budget	incident/task ID
request_human_action	Human	operator ID + start/end + outcome
open_owned_test_app	Automatic in lab only	test ID
third_party_social_ui_action	Human by default	authorization + operator audit; no autonomous macro library
account_creation / bulk friend-add / fake engagement / enforcement bypass	Blocked	no implementation

5.4 Device lifecycle

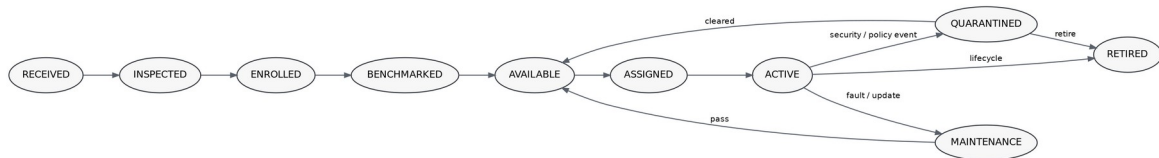


Figure A3 - Device lifecycle state machine.

Do not overload “banned,” “broken,” “offline,” and “retired.” They are different states with different responses. A platform enforcement event belongs to the account/device incident record; it must not automatically trigger account replacement. A hardware fault belongs to maintenance. A security concern moves the device/node to quarantine. Retirement is a deliberate asset lifecycle action.

6. Physical, mechanical and electrical specification

6.1 Intact handset versus bare motherboard

For the durable pod, intact handsets are the default. They retain camera/SIM/sensors, simplify inspection, preserve resale/service value and keep the device in a vendor-supported mechanical configuration. Bare-board chassis can offer density, but they change thermal behavior, residual value, sensor availability, cable strain and servicing. Only consider bare boards for a separate purpose-built lab after the durable handset pod is stable and there is a quantified density benefit.

6.2 Chassis acceptance checklist

- Every slot has a permanent rack/slot ID visible from the service side.
- Phones can be removed without disturbing adjacent control boards/cables.
- No handset is clamped across a swollen or deformable battery region.
- Air intake and exhaust are unobstructed; cables do not form a thermal blanket.
- Fans are replaceable and their failure can be detected visually or via node inspection.
- Power supply rating, input voltage, DC rails and per-port limits are documented.
- Control boards are strain-relieved; Lightning/USB-C connectors are not load-bearing.

- Chassis has an equipment-ground path where required and uses certified mains components appropriate to jurisdiction.
- An emergency power-off method is obvious and accessible.
- The rack can be serviced without defeating fire detection or blocking airflow.

6.3 Power-budget method

Do not budget from the charger label alone and do not assume every device draws its maximum simultaneously. Measure the pod. Use the following design equation as a planning envelope:

$$\begin{aligned} \text{Pod_design_W} = & 1.5 \times \text{measured_peak_device_and_board_W} \\ & + \text{measured_host_peak_W} \\ & + \text{fans_and_network_W} \\ & + \text{future_headroom_W} \end{aligned}$$

The 1.5 multiplier is an engineering margin used here for the measured phone/control-board portion; it is not an Apple or USB-IF requirement. Replace it if your electrical engineer or measured inrush/charging data supports another margin. The key is that “20 phones” is not a wattage.

6.4 Charging policy for durable devices

A durable set should avoid keeping old batteries hot at 100% charge indefinitely. Use supported charge-optimization/charge-limit features where available and, where the hardware permits, managed per-port power or scheduled charging. More important than a specific percentage is thermal headroom and observability. Apple states that high ambient temperature can permanently shorten battery life and that charging can slow or pause when the device is too warm.

Sources: [P18-P19]

6.5 Thermal design targets

Parameter	Starting target	Reason
Room / rack intake	20-26 C design target	Provides margin under Apple's 35 C upper operating limit; local engineering target, not Apple requirement.
Alarm level	30 C ambient intake or abnormal device thermal warnings	Early intervention before the 35 C operating boundary.
Spacing	Enough for unobstructed front/back convection; verify in thermal test	Do not rely on a fixed mm value without the actual chassis.
Soak test	At least one full work-cycle and one extended idle/charge cycle	Catches charging heat and fan/controller failures.
Inspection	Battery swelling, connector discoloration, fan noise, unusual power draw	Remove suspect devices from service immediately.

6.6 Host-computer starting specification

For a 20-device iMouse pod, the vendor's minimum Windows requirements are too weak to use as a production sizing rule. Start with a dedicated modern Windows 11 Pro workstation: 8-core-or-better CPU, 32 GB RAM, 1 TB NVMe, hardware-accelerated graphics, dual 1 GbE or faster NIC capability, and enough independent USB controllers/ports for the chosen chassis interface. This is a starting BOM, not a guaranteed 20-device capacity; TEST-CAPACITY-001 decides the final host.

A Mac mini is not the default host for iMouse because the vendor currently publishes Windows-only support. A Mac remains relevant for Apple-native development/testing (Xcode, WebDriverAgent/Appium, Configurator) but should not be purchased as the production controller until the exact control product is selected.

Sources: [V01-V04]

7. Network architecture, cellular egress, proxies and VPNs

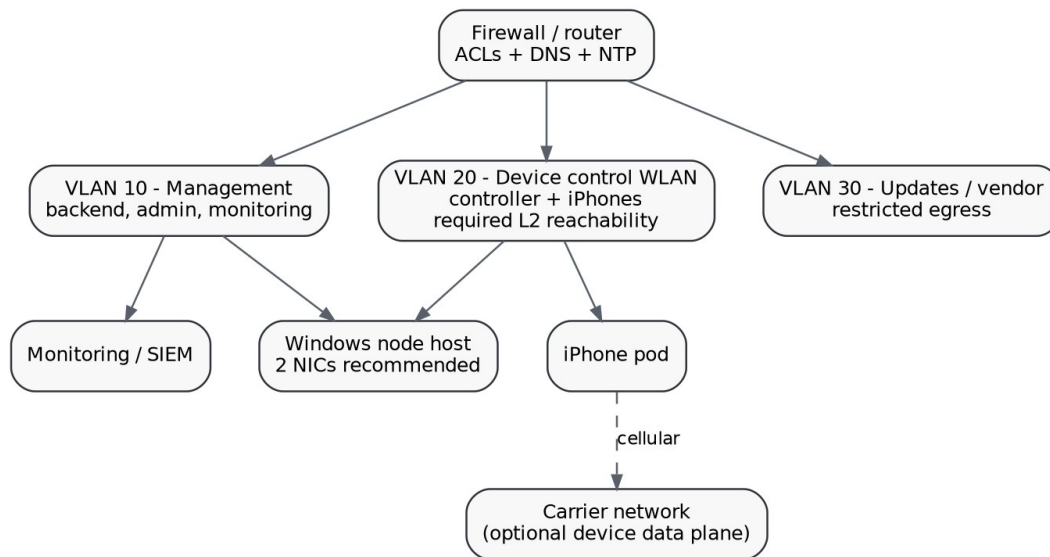


Figure A4 - Network segmentation starting point.

7.1 Separate control-plane networking from Internet egress

The key architectural mistake to avoid is treating “the phone’s IP” as one property. Each phone can simultaneously participate in local control-plane traffic, DNS, Wi-Fi Internet traffic, cellular traffic and managed VPN traffic. The design must state which interface is intended for which function and then verify the actual path.

7.2 Recommended VLAN / zone model

Zone	Contains	Allowed flows
Management	Backend, database, monitoring, admin workstations	Admin -> API; backend -> node-agent; no phone broadcast domain
Device control WLAN	iPhones + corresponding Windows node interface	Phone <-> node local mirroring/control discovery; restricted access to other zones
Vendor/update zone	Controlled egress for installer/update/licensing if required	Node -> explicit vendor destinations; deny unsolicited inbound
Object/media service	Approved media/evidence storage	Node/backend authenticated access
Guest/employee	Ordinary staff devices	Dashboard only; never vendor control ports
Cellular	Carrier interface on each phone where used	Separate data plane; observed, not assumed

7.3 Forward proxy fundamentals

A forward proxy accepts application connections on behalf of a client. HTTP proxies typically proxy HTTP and use CONNECT for HTTPS tunnels; SOCKS5 is more protocol-agnostic. A VPN/TUN configuration instead changes the device’s routing so IP traffic enters a tunnel. Carrier NAT/CGNAT is neither a proxy nor a VPN: it is address translation inside the carrier network. These mechanisms solve different problems.

When a proxy is legitimate in this architecture

- Corporate egress control, logging and content filtering.
- Testing your own service from a known network path or region where terms permit.
- Segregating internal application traffic from the general office network.
- Enforcing organizational policy for managed apps or devices.

What is not a design requirement

- “One proxy per device” as a universal anti-ban rule.
- Rotating IPs to make coordinated accounts appear unrelated.
- Hiding actual location to manipulate a recommendation system.
- Using residential/mobile proxy labels as a substitute for platform authorization.

7.4 Proxy vendor evaluation sheet

Dimension	Questions
Protocol	HTTP CONNECT, SOCKS5, WireGuard/OpenVPN, vendor app?
Authentication	Per-device credential, certificate, IP allow-list? Rotation process?
DNS	Where is resolution performed? IPv6 behavior? DNS leak test for your own endpoint?
Logging/privacy	What metadata is retained, for how long, and in which jurisdiction?
Address class	Datacenter/residential/mobile; is resale/traffic sourcing lawful and contractually clear?
Capacity	Sustained Mbps, concurrent connections, latency, packet loss, quotas.
Failure mode	Fail closed/open? What does the phone do if tunnel/proxy drops?
Management	API, usage telemetry, credential revocation, invoices, data-export.
Terms	Does the provider permit your intended business use?
Security	Client code provenance, root CA requirements, device profile privileges.

7.5 Apple managed VPN options

Apple’s current deployment documentation supports per-app VPN for managed apps and Always On VPN for supervised devices. Always On can create separate tunnels for cellular and Wi-Fi interfaces. These are enterprise security features. If used, configure them for organizational security/observability, not for concealment. Because per-app VPN requires an app to be managed by MDM, the exact deployment method must be confirmed for any App Store app in scope.

Sources: [P11-P12]

7.6 Egress observability record

```

egress_observation:
  device_id: ios-pod1-slot03
  observed_at: 2026-08-26T18:00:00Z
  control_wifi:
    ssid: PF-CONTROL-01
    local_ip: 10.20.20.43
    local_mirror_ok: true
  cellular:
    carrier: <carrier>
    enabled: true
  neutral_endpoint_source_ip: <observed>
  router_wan_bytes_delta: <measured>
  device_cellular_bytes_delta: <measured>
  conclusion: <wifi|cellular|mixed|unresolved>
  evidence_refs: [pcap_or_counter_ref, endpoint_log_ref, screenshot_ref]
```

8. iOS fleet management and control-stack choices

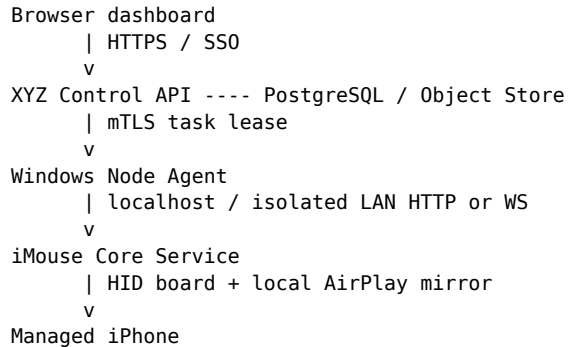
8.1 Keep MDM and remote-control tooling separate

MDM manages organizational policy: enrollment, supervision, restrictions, managed apps, Wi-Fi/VPN profiles, update policy, inventory and remote administrative actions. iMouse is a local interactive-control/mirroring product. Do not make one pretend to be the other. A production pod can use both: MDM for fleet governance and iMouse for local human remote control/media operations.

Apple's July 2026 Platform Deployment guide supports organization-owned device management through Apple Business Manager plus a device-management service. Automated Device Enrollment can supervise owned iPhones and keep management configuration in place through lifecycle events. This is the durable-fleet path.

Sources: [P20-P23]

8.2 iMouse integration pattern



The node agent is an anti-corruption layer around the vendor API. The rest of the system should not know vendor endpoint names, board IDs, or undocumented quirks. If iMouse is replaced later, only the adapter changes.

8.3 Security controls around iMouse

- Dedicated Windows account and machine; no web browsing, email or personal developer work on the node.
- Windows firewall defaults to deny inbound; allow only the control WLAN/mgmt flows required.
- Never NAT/port-forward the vendor HTTP/WebSocket/AirPlay service to the public Internet.
- Use VLAN ACLs so employee/guest networks cannot reach vendor ports.
- If vendor requires antivirus exclusions, make the smallest process/path exclusion possible, document it and compensate with network isolation and file-integrity monitoring. Do not uninstall endpoint protection as routine setup.
- Disable vendor auto-update in production until updates are tested on a staging node if the product allows it; otherwise schedule controlled maintenance and hash/version capture.
- Export or record configuration needed for disaster recovery, but do not store vendor credentials inside source control.
- Log node-agent calls at the semantic level (device, approved task, outcome), not raw platform credentials or sensitive media content.

8.4 Apple-native alternatives: where they fit

Tool/path	Strength	Constraint	Best use here
XCUITest / WebDriverAgent / Appium	Official/standard test automation ecosystem	Real-device setup depends on macOS/Xcode, Developer Mode, signing/provisioning and WDA lifecycle	QA of owned apps; test lab, not default 100+ social-phone controller
Apple Configurator	Enrollment/preparation utility	Mac-centric; not a live remote-control dashboard	Supervision/enrollment support
MDM	Fleet policy and inventory	Not arbitrary interactive UI control	Production governance
iMouse	No jailbreak/signing/on-device app; hardware control; browser/API	Windows-only vendor dependency; same-LAN; security and input semantics require testing	Candidate human remote console for durable pod
Screen mirroring alone	Low control complexity	May not provide input or API	Operator viewing / diagnostics

8.5 Reject the “freeze iOS 16 to hide pointer type” production strategy

The internal brief raises a possible old-OS freeze to preserve compatibility-mode touch behavior. Do not adopt this as a production design. Its motivating purpose is to avoid an observable input semantic; it creates security-patch, app-compatibility, replacement-stock and accidental-upgrade fragility. Run current supported operating systems and design the workflow so it does not rely on disguising the control method.

9. Android alternative architecture

9.1 Why Android remains operationally attractive

Android has a mature local engineering toolchain. ADB is the standard debug bridge, and scrcpy can mirror/control devices over USB or TCP/IP without root; current scrcpy documentation describes Windows, Linux and macOS support and 30-120 fps depending on the device. Android Enterprise also supports fully managed and dedicated company-owned devices through policy-driven enrollment. This makes Android substantially easier to integrate into an auditable fleet without proprietary per-phone control boards.

Sources: [P24-P27]

9.2 Recommended Android pod pattern

- Company-owned, stock, supported Android devices; no root or custom ROM requirement.
- Android Enterprise fully managed/dedicated enrollment where appropriate.
- ADB/scrcpy reserved for authorized engineering and controlled local operation.
- Same XYZ backend, task model, inventory schema, object storage and audit layer as iOS.
- Platform-specific node adapter (`android-adapter`) behind the same semantic task interface.
- No assumption that a factory reset removes all server-side integrity history.

9.3 Android identifier reality

Mechanism	Current project interpretation
IMEI / serial	Restricted for ordinary apps on modern Android; do not use vendor folklore that assumes arbitrary access.
MAC address	Third-party access is restricted; randomization/privacy mechanisms apply.
Android/SSAID	Scoped/reset behavior depends on app/user/device context; not a universal immutable hardware ID.
Play Integrity	Can attest device/app integrity and other risk signals.
Device recall beta	Three developer-defined per-device values can survive reinstall and reset on Google servers.
Project rule	A wipe is maintenance; never promise it “cleans” device reputation.

9.4 iOS versus Android decision matrix for the durable set

Criterion	iOS + iMouse	Android + Enterprise/ADB	Implication
Current client ecosystem	Likely strong if existing workflow is iPhone-specific	Depends on app/content workflow	Do not switch solely for engineering convenience
Central interactive control	Vendor hardware/software	Open ADB/scrcpy options	Android simpler
Per-phone control hardware	Yes for iMouse	No for ADB path	Android lower BOM
Fleet policy	ABM/MDM strong	Android Enterprise strong	Both viable
Media import	Vendor/Shortcuts/Photos route	ADB/media scanner easier	Android simpler
Host OS	Windows for iMouse; Mac for Apple-native test tools	Windows/Linux/macOS	Android flexible
Reset-surviving anti-abuse state	Apple DeviceCheck capability exists	Play Integrity device recall beta exists	Neither should be treated as clean-slate reset
Recommendation	Use when iOS is an actual business requirement	Use for engineering-heavy controlled fleets	Pilot both only if platform strategy justifies

10. Software control plane - concrete implementation specification

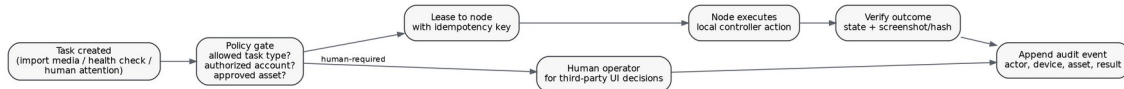


Figure A5 - Safe task execution flow with policy gate and human path.

10.1 Reference stack

Layer	Recommended technology	Reason
Backend API	Python 3.12 + FastAPI + Pydantic	Matches iMouse Python ecosystem, strong typed APIs, rapid integration.
ORM/migrations	SQLAlchemy 2 + Alembic	Mature Postgres integration and explicit schema migration.
Database	PostgreSQL 16+	Durable source of truth, JSONB where needed, strong constraints.
Ephemeral queue/cache	Redis 7+	Task leases, heartbeats, short-lived coordination; not source of truth.
Frontend	Next.js + TypeScript	Operations dashboard, SSO integration, typed client.
Object storage	S3-compatible (cloud S3 or MinIO for local pilot)	Media/evidence storage with lifecycle policies.
Node agent	Python Windows service	Local vendor adapter and telemetry collector.
Metrics	Prometheus/OpenTelemetry-compatible metrics	Standard fleet/node observability.
Logs	Structured JSON -> centralized collector	Searchable incident evidence.
Secrets	Cloud secret manager or Vault-class service	No secrets in DB/source/config files.
Packaging	Docker for backend services; native Windows service for hardware node	USB/AirPlay vendor controller should remain native on node.

10.2 Repository layout

```

phone-farm-control/
  README.md
  SECURITY.md
  docs/
    architecture/
    decisions/          # ADR-0001 ...
    tests/              # TEST-IOS-INPUT-001 ...
    runbooks/
    source-ledger.md
    unknowns.md
  backend/
    app/
      api/
        domain/
        services/
        adapters/
        models/
        security/
      migrations/
      tests/
  dashboard/
  node-agent/
    adapters/
    imouse/
    android/
    telemetry/
    service/
    tests/
  infra/
    docker/
    monitoring/

```

```
network/  
schemas/  
events/  
api/  
scripts/  
dev/  
ops/
```

10.3 Architectural boundaries

- `domain/` never imports the iMouse Python package. It speaks in semantic operations such as `ImportMedia`, `CaptureDiagnosticScreenshot`, `GetDeviceHealth` and `RequestHumanAttention`.
- `node-agent/adapters/imouse` is the only module that knows iMouse endpoint names, board IDs and vendor response formats.
- `node-agent/adapters/android` uses ADB/scrcpy only for authorized engineering/device operations.
- Third-party platform credentials are referenced via secret IDs and authorization records, not embedded in tasks.
- Every task is idempotent or explicitly non-retryable; the system never repeats a social/publishing action merely because the network response was lost.
- Every state mutation appends an audit event; mutable “current state” is derived from authoritative tables plus events.

10.4 Device data model

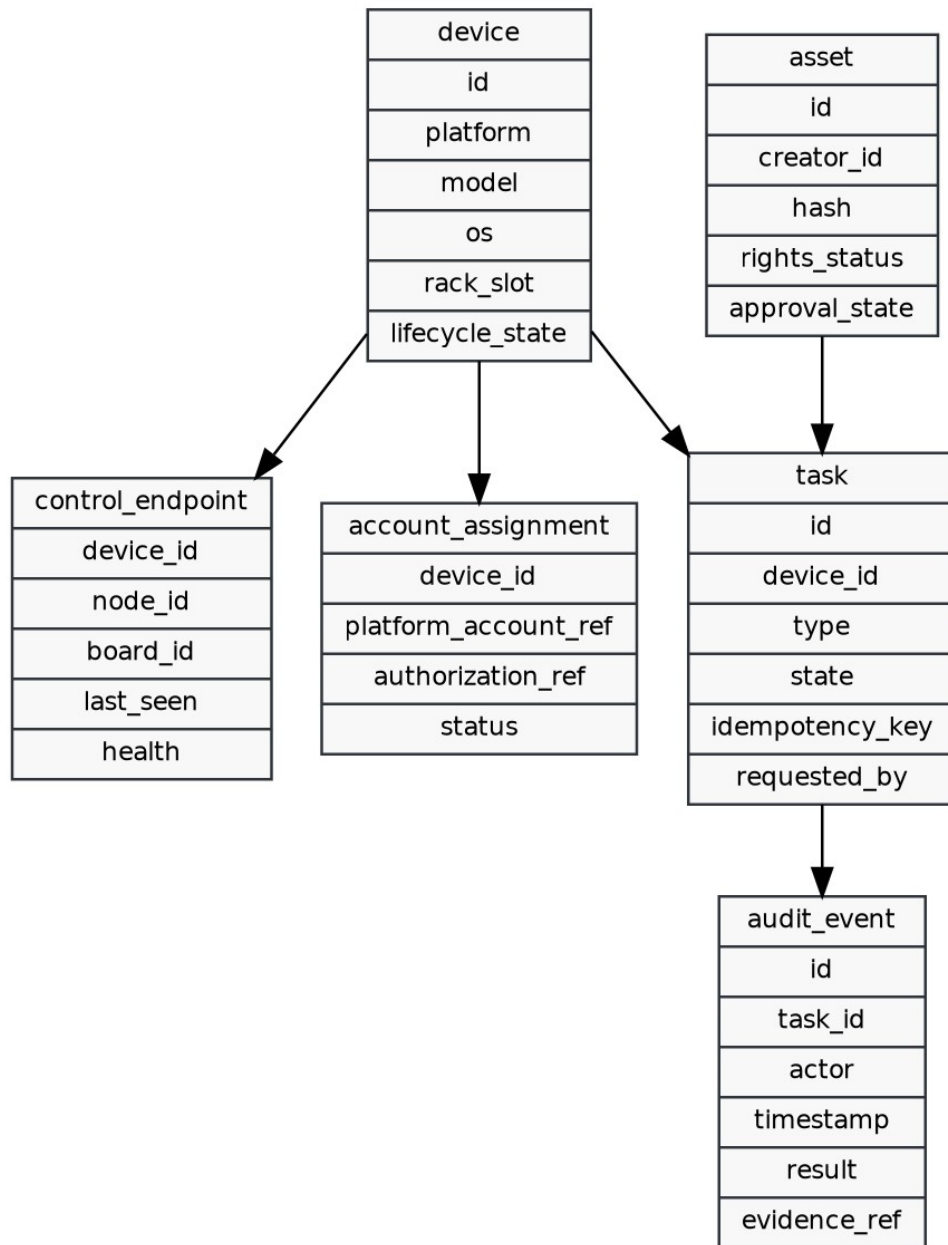


Figure A6 - Minimal entity relationships.

```

device
  id UUID PK
  asset_tag TEXT UNIQUE
  platform ENUM(iOS, Android)
  model TEXT
  os_version TEXT
  serial_ref TEXT NULL      # internal inventory ref, not exposed broadly
  rack_id UUID
  slot_no INT
  lifecycle_state ENUM
  mdm_ref TEXT NULL
  acquired_at TIMESTAMPTZ
  retired_at TIMESTAMPTZ NULL
  
```

```

control_endpoint
  
```

```

device_id UUID FK
node_id UUID FK
adapter ENUM(imouse, android)
vendor_device_ref TEXT
board_ref TEXT NULL
last_seen_at TIMESTAMPTZ
health_state ENUM

```

```

account_assignment
  id UUID PK
  device_id UUID FK
  platform TEXT
  account_external_ref TEXT
  creator_id UUID
  authorization_ref TEXT
  status ENUM(active, paused, review, ended)
  secret_ref TEXT NULL

```

```

asset
  id UUID PK
  creator_id UUID
  sha256 TEXT UNIQUE
  object_key TEXT
  media_type TEXT
  rights_status ENUM
  approval_state ENUM
  safety_classification TEXT

```

```

task
  id UUID PK
  type TEXT
  device_id UUID NULL
  asset_id UUID NULL
  state ENUM(queued, leased, running, human_wait, succeeded, failed, cancelled)
  idempotency_key TEXT UNIQUE
  requested_by UUID
  policy_decision JSONB
  lease_expires_at TIMESTAMPTZ NULL

```

```

audit_event
  id BIGSERIAL PK
  occurred_at TIMESTAMPTZ
  actor_type TEXT
  actor_id TEXT
  event_type TEXT
  entity_type TEXT
  entity_id TEXT
  result TEXT
  evidence_ref TEXT NULL
  metadata JSONB

```

10.5 Task API examples

```

POST /v1/tasks
{
  "type": "import_media",
  "device_id": "...",
  "asset_id": "...",
  "idempotency_key": "import:<device>:<asset>:v1"
}

```

```

POST /v1/tasks
{
  "type": "request_human_action",
  "device_id": "...",
  "reason": "Publish approved item using authorized account",
  "approval_ref": "APR-2026-..."
}

```

```
GET /v1/devices?rack_id=...&health=degraded
GET /v1/tasks/{id}
POST /v1/tasks/{id}/cancel
GET /v1/audit?device_id=...&from=...&to=...
```

10.6 Policy engine

Before a task can be leased to a node, evaluate a small explicit policy. This is not a generic AI judgment. It is deterministic code backed by authorization and asset state.

```
allow(task) only if:
    device.lifecycle_state in {AVAILABLE, ASSIGNED, ACTIVE}
    AND node.health == HEALTHY
    AND task.type in ALLOWED_TASK_TYPES
    AND (asset is null OR asset.approval_state == APPROVED)
    AND (account is null OR account.authorization_ref is valid)
    AND no quarantine/security hold exists
    AND any human-required task has assigned operator

BLOCKED_TASK_TYPES include:
    bulk_account_creation
    bulk_follow_or_friend_add
    fake_engagement
    fingerprint_spoofing
    enforcement_bypass
    automatic_replacement_after_platform_ban
```

11. Node agent and vendor-adapter design

11.1 Node-agent responsibilities

- Authenticate to backend with a unique node certificate and rotate credentials.
- Register hardware/software version, node health, NICs and adapter capabilities.
- Lease only tasks assigned to its pod and renew leases while executing.
- Validate task schema and local policy before calling the vendor adapter.
- Stage approved media in an ephemeral local directory; verify SHA-256 before import.
- Collect adapter/device health and send structured heartbeats.
- Upload diagnostic evidence through signed object-storage URLs.
- On restart, reconcile leased tasks rather than blindly repeating them.
- Never persist platform passwords in logs, screenshots directory names or crash dumps.

11.2 Adapter interface

```
class DeviceAdapter(Protocol):
    def list_devices(self) -> list[DeviceStatus]: ...
    def get_health(self, device_ref: str) -> DeviceHealth: ...
    def capture_diagnostic_screenshot(self, device_ref: str) -> bytes: ...
    def import_media(self, device_ref: str, local_path: str) -> ImportResult: ...
    def restart_control_channel(self, device_ref: str) -> RestartResult: ...

    # Deliberately not part of the generic production interface:
    # arbitrary_click_macro()
    # bulk_account_create()
    # auto_follow_users()
    # rotate_identity_after_ban()
```

11.3 Heartbeat contract

```
{
    "node_id": "pod-01-node",
    "timestamp": "2026-08-26T18:10:00Z",
```

```

"agent_version": "0.1.0",
"adapter": {"name": "imouse", "version": "<measured>"},
"host": {"cpu_pct": 32.4, "ram_pct": 41.0, "rx_mbps": 18.2, "tx_mbps": 4.1},
"devices": [
  {"device_id": "ios-pl-s01", "connected": true, "mirror": true, "last_frame_ms": 85},
  {"device_id": "ios-pl-s02", "connected": true, "mirror": true, "last_frame_ms": 91}
]
}

```

11.4 Retry semantics

Operation	Retry policy
Read health/list devices	Safe exponential retry with bounded timeout.
Capture diagnostic screenshot	Safe bounded retry; dedupe evidence by task ID.
Import media	Retry only if adapter provides deterministic success check; verify Photos presence/hash surrogate where possible.
Restart local service	One automatic attempt, then human/incident escalation.
Human third-party UI action	Never auto-retry; operator must confirm outcome.
Unknown vendor error	Fail closed and preserve raw vendor error in restricted diagnostic field.

11.5 Local security boundary

The node agent should connect outward to the backend; the backend should not open arbitrary inbound administrative ports on the Windows node. If inbound management is required, use a corporate VPN/zero-trust access layer with device identity and MFA. Keep iMouse itself bound to localhost or the control VLAN as narrowly as the product permits. Use the vendor `allow\_ip\_list` feature if validated, but do not treat it as the sole security control.

12. Content, account authorization and human operations

12.1 Content pipeline

```

Creator / rights holder
-> ingest
-> hash + metadata
-> safety / rights review
-> approval
-> variant generation (if authorized)
-> campaign assignment
-> device import queue
-> human publishing / official API
-> result capture
-> attribution + retention

```

12.2 Required asset metadata

Field	Purpose
creator_id	Ownership/authorization boundary.
sha256	Deduplication and evidence of exact asset used.
source_ref	Original storage/source.
rights_status	Owned/licensed/expired/restricted.
approval_state	Prevents unreviewed content from reaching a device queue.
safety_classification	Internal routing and platform-policy review.
campaign_id	Attribution.
variant_of	Tracks crops/edits/captions as derivatives.
created_by / approved_by	Audit trail.
retention_class	Controls deletion on creator exit or campaign end.

12.3 Account authorization model

The system should never infer that an account is authorized because credentials work. Maintain an explicit authorization record: creator/owner, platform, account identifier, date, scope, permitted staff roles and revocation. If the organization cannot produce that record, pause the assignment.

12.4 Human operator screen

- Device thumbnail and health status.
- Assigned authorized account and creator identity.
- Approved asset/caption/link reference.
- Clear action objective in plain language.
- Button to start a human session, taking an audit lock on that device.
- Outcome choices: completed / skipped / blocked / platform warning / needs manager.
- Optional diagnostic screenshot with explicit retention and privacy handling.
- No bulk “run on all phones” button for third-party social actions.

12.5 The “five-post threshold” test as a product experiment

Represent the internal observation as a hypothesis, not a workflow constant. A clean experiment could compare authorized test accounts assigned to predefined content counts while holding other known variables as consistent as practical. Predeclare the primary metric (e.g., public-profile impressions or link clicks), the observation window and the stop conditions. Do not use scraped-user adding, ban cycling or other confounded tactics in the experiment; the goal is to isolate whether profile content volume correlates with distribution.

13. Platform policy and operational boundary

A technically stable fleet can still be a failed system if the traffic method violates platform rules. Treat platform policy as a functional requirement. The policy module should be reviewed at least quarterly and before any major new traffic strategy.

13.1 Snapchat

- Snap documents SS06/SS18 device bans for abuse/repeated violations and SS07 when too many accounts are associated with a device.
- Snap says banned devices cannot create new accounts; appeal is through its approved in-app process.
- Snap’s March 2026 sexual-content explainer prohibits offers or solicitation of sexual services, including online sexual subscription services.
- Public profile names/content can also be restricted if they reference prohibited commercial sexual transactions or spam.
- Architecture consequence: do not code automated device/account replacement as a response to platform enforcement; do not assume a SFW creative makes a prohibited destination permissible.

Sources: [P01-P03]

13.2 TikTok

- TikTok’s current guidelines prohibit deceptive behavior/fake engagement, and its published materials state that automation used to register or operate accounts in bulk is not allowed.
- TikTok can act against attempts to avoid enforcement and against multiple accounts used to dodge rules.
- Use approved APIs for integrations where available and comply with app-review/developer requirements.
- Keep third-party UI operation human unless a specific authorized automation path is documented by the platform.

Sources: [P28-P31]

13.3 Reddit

- Reddit Rule 2 requires authentic participation and prohibits spam/disruptive behavior including content manipulation.
- The July 2026 User Agreement prohibits automated or other use that disrupts, overburdens or impairs the service; commercial API/data use has separate terms.
- Use subreddit rules and approved API access as first-class constraints; do not treat physical phones as permission to bypass API/commercial terms.

Sources: [P32-P34]

13.4 Instagram / Meta and other platforms

Platform rules and supported publishing interfaces change frequently. Part B contains broader operational notes. Before implementation, Claude should retrieve the current first-party Terms/Community Standards/API documentation for every platform actually selected and add them to `docs/source-ledger.md`. Do not rely on an OFM vendor's summary of a platform rule when a first-party source is available.

14. Observability, audit, security and incident response

14.1 What must be observable

Layer	Minimum metrics / events
Physical	pod power draw, fan status where available, environmental temperature, device charging/thermal warnings
Windows node	CPU, GPU, RAM, disk, NIC, service state, reboot reason, agent version
Control adapter	connected devices, mirror state, reconnects, API errors, frame latency/quality if exposed
Device	last seen, OS version, storage headroom, MDM compliance, assigned account
Task system	queue depth, lease age, success/failure, retries, human wait time
Media	import latency, hash verification, duplicate/rejected asset counts
Security	failed auth, blocked task types, privilege changes, quarantines, secret access
Business	approved posts, link clicks/conversions where lawfully measured, cost per outcome

14.2 SLO starting points

These are internal engineering starting points, not guarantees. Revise after the 5-device pilot.

SLO	Starting target
Backend API availability	99.5% during pilot operations
Node heartbeat freshness	95% of heartbeats arrive within 2x interval
Inventory correctness	100% of physical slots mapped to a device or explicitly EMPTY
Audit coverage	100% of task state changes produce audit event
Approved-media integrity	100% SHA-256 verified before node import
Unauthorized task execution	0; blocked task attempts logged
Recovery from node-agent restart	No orphan task is silently repeated
RPO for database	<= 15 minutes pilot; tighten for production
RTO for backend	<= 4 hours pilot; pod can be safely paused meanwhile

14.3 Security threat model

- Compromised Windows vendor software or updater.
- Unauthenticated local vendor API exposure.
- Employee misuse of remote-control capability.
- Credential/media leakage from screenshots/logs.

- Cross-creator data exposure in queues or storage.
- Ransomware or node failure causing operational outage.
- Unsafe charging/battery failure in dense racks.
- Platform warning ignored and multiplied across accounts.
- Shadow IT proxy/VPN apps installed without review.
- AI-generated code that silently adds a prohibited generic macro/automation endpoint.

14.4 Incident classes

Class	Immediate action	Then
SEC - node compromise	Isolate node VLAN, stop agent/vendor service, revoke node certificate	Preserve logs/image; rebuild from known-good image
PHY - swollen/hot battery	Remove power and isolate device safely	Replace device; inspect neighboring slot/power port
NET - route/egress anomaly	Pause affected pod tasks	Collect counters/routes; do not change proxy identities blindly
PLAT - account/device enforcement	Pause assignment and escalate	Review platform notice/policy; no automatic replacement
DATA - wrong creator asset	Stop affected queue and revoke object URLs	Audit assignments; notify responsible owner; remediate retention
CTRL - iMouse instability	Stop new tasks, restart controlled service once	Capture version/logs; fail over to manual/local handling if safe

15. Build plan and stage gates



Figure A7 - Scale only after evidence gates pass.

Gate 0 - inventory and architecture intake

1. Meet with Dim and capture the exact tested control software, version, installer source, screenshots/video, hardware board, cable topology and observed limits.
2. Inventory all phones: model, OS, battery condition, lock/ownership state, SIM capability, connector, MDM eligibility.
3. Inventory the 20-slot enclosure: vendor/model, PSU label, fans, board types, port topology, required host connection.
4. Create `docs/unknowns.md` with every unresolved claim from the internal brief.
5. Create ADR-0001 "Durable pod first" and ADR-0002 "No autonomous third-party social UI automation."

Gate 0 exit

No hardware purchase is approved until the existing inventory is reconciled and the actual Dim-tested software is identified.

Gate 1 - two-device evidence bench

6. Run TEST-IOS-INPUT-001.
7. Run TEST-NET-EGRESS-001.
8. Run TEST-IOS-CONTROL-001.
9. Run TEST-MEDIA-IOS-001.
10. Capture Windows/iMouse ports, service dependencies and security exceptions.
11. Decide iMouse vs Dim-tested alternative with ADR-0003.

Gate 2 - five-device pilot pod

12. Build the backend skeleton and PostgreSQL schema.
13. Implement node registration/heartbeats and `imouse` adapter for health, screenshot and approved media import.
14. Build device inventory UI and physical slot mapping.
15. Implement creator/account authorization references and asset approvals.
16. Implement human session lock + outcome reporting.
17. Run 5-device soak and collect power/thermal/network/controller metrics.
18. Complete security review before storing real creator media.

Gate 3 - ten-device stable pod

19. Expand only by duplicating the known working wiring/slot pattern.
20. Run capacity tests and node restart/recovery drills.
21. Add backup/restore, alerts, operator roles and offboarding.
22. Measure actual operator time per approved publish task and media-import latency.
23. Review platform rules again and freeze the pilot operating SOP.

Gate 4 - twenty-device production pod

24. Complete full chassis thermal/power soak.
25. Document spare ratio and replacement procedure.
26. Enforce SSO/MFA and production secret management.
27. Add business attribution dashboards only from authorized tracking data.
28. Complete go/no-go review for a second pod. Do not jump to 100+ because one 20-device pod works.

Gate 5 - multi-pod scale

Scale horizontally: pod-01, pod-02, pod-03. Each pod has a unique node certificate, control subnet/SSID identity, rack IDs, monitoring labels and failure domain. Central services schedule only to healthy pods. Keep a staging pod for controller/vendor updates before production rollout.

16. Acceptance criteria by subsystem

16.1 Physical pod

- Every slot, phone, control board, cable group and power output is labeled and represented in inventory.
- Full-load thermal soak completes without thermal charging hold, device shutdown, abnormal battery behavior or unplanned controller disconnect attributable to heat.
- Power draw is measured at idle, active mirroring and simultaneous charging; PSU and upstream circuit have documented headroom.
- One phone can be removed/replaced without disturbing neighboring devices.
- Emergency shutdown and battery incident procedure is posted and tested.

16.2 Windows/iMouse control node

- Node can cold boot and restore all healthy device mirrors without manual undocumented steps.
- Vendor service ports are unreachable from guest/employee networks and the public Internet.
- Endpoint security remains enabled with documented narrow exceptions only if needed.
- Agent can detect vendor service failure, attempt one controlled restart, then escalate.
- No vendor API credential or node certificate is committed to source control.
- A second engineer/operator can reproduce installation from the runbook.

16.3 Backend/control plane

- PostgreSQL is the source of truth; Redis loss does not lose inventory or completed audit history.
- Duplicate task submissions with same idempotency key do not create duplicate effects.
- Every task outcome is attributable to actor, device, asset and authorization context.
- Blocked task types cannot be created by normal API clients.
- RBAC prevents a staff member assigned to creator A from browsing creator B media unless explicitly authorized.
- Backup restoration is tested, not merely configured.

16.4 Operations

- An operator can identify which device needs attention in under one minute from the dashboard.
- No one needs a spreadsheet to determine current phone/account/slot ownership.
- Approved content reaches the intended phone with hash/asset traceability.
- Platform warnings are captured as incidents and stop further automated internal tasks for the affected assignment until review.
- Creator exit can revoke account assignment and media access without dismantling the whole fleet.

17. Cost model and procurement framework

The internal brief contains modeled 200-device economics. Preserve those numbers as scenarios, not facts. The more useful immediate model is pod-level total cost of ownership with measured quantities. The first purchasing decision is not “iOS or Android in the abstract”; it is “what does one stable, supported 20-device pod cost to acquire, run, staff and maintain for 12 months?”

17.1 TCO worksheet

```
12_month_TCO =
  phones_and_spare
+ chassis_and_control_boards
+ Windows_node_or_controller
+ network_AP_switch_firewall_share
+ MDM_or_management_licenses
+ controller_software_licenses
+ cellular_or_ISP_cost
+ storage_and_backend_share
+ power_cost
+ replacement_parts
+ operator_labor
+ engineering_maintenance
- expected_residual_value
```

17.2 Procurement fields

Item	Record before approval
Phone	model, OS support, storage, battery health, lock status, connector, SIM/eSIM, condition, unit price, warranty/return
iMouse board	connector type, unit price, license, vendor source, firmware/version, spare rate
Chassis	slot count, intact-handset support, PSU watts, fan count, host interface, dimensions, delivery
Windows node	CPU/GPU/RAM/NVMe/NIC/USB controllers, warranty, OS license
Networking	AP radios, switch ports/PoE, firewall, VLAN support, UPS
MDM	per-device/month, ABM integration, supervised controls, API/export
Carrier	line activation, data policy, tethering restrictions, pooled vs per-line, cancellation
Proxy/VPN if used	contractual purpose, protocol, logging, bandwidth, pricing, security review

17.3 Current product-price notes

Vendor pages observed during this research list the iMouse control board at roughly USD 38.33 retail and a vendor-stated recurring iMouse license around USD 0.75 per phone/month after a trial. Rack/chassis prices vary substantially by configuration. Treat all prices as volatile snapshots; re-quote before purchase and do not infer bulk pricing from single-unit pages.

Sources: [V01, V06]

18. Risk register

ID	Risk	Likelihood	Impact	Mitigation / decision
R-01	Control vendor requires unsafe endpoint-security exclusions	M	H	Dedicated node, narrow exception, isolation, staging, hashes; reject if containment inadequate.
R-02	iMouse input is distinguishable in actual app path	M	M/H	Measure in owned test app; do not rely on invisibility; human-only use or alternate control.
R-03	Control WLAN captures Internet egress unexpectedly	M	M	Run egress test; choose documented routing; never assume SIM.
R-04	Platform app raises minimum OS and strands old fleet	H over lifecycle	H	Buy current-supported models; update ring; app compatibility watch.
R-05	Battery heat/swelling in dense rack	M	H	Thermal design, inspection, managed charging, power telemetry, remove damaged cells.
R-06	Vendor controller instability at 20+	M	M/H	Capacity test; pod isolation; horizontal scale.
R-07	Operator claim drives over-purchase before replication	M	H	Stage gates and n=5/n=10 evidence; no 100+ procurement from testimony.
R-08	Reset assumed to erase platform device history	M	H	Document DeviceCheck/device recall reality; treat enforcement as incident, not reset workflow.
R-09	Media/credential cross-creator leak	M	H	RBAC, separate prefixes, secret manager, audits, creator exit procedure.
R-10	AI coding agent adds prohibited generic automation	M	H	Blocked task list, code review, policy tests, SECURITY.md.
R-11	Platform terms change	H	M/H	Quarterly source refresh + pre-launch review.
R-12	Single Windows node becomes fleet-wide blast radius	M	H	20-device pods, separate certificates/segments, staged updates.

19. Architecture Decision Records Claude must create

ADR	Decision	Required evidence before closing
ADR-0001	Durable pod first vs disposable fleet	Internal readiness + platform policy + operator evidence classification.
ADR-0002	Third-party UI automation boundary	Platform rules and project safety boundary.
ADR-0003	Dim-tested controller vs iMouse	Hands-on comparison, security, API, stability,

ADR	Decision	Required evidence before closing
		host OS, cost.
ADR-0004	iOS device acquisition floor	Current Apple OS support + current app compatibility + price.
ADR-0005	20 devices per pod	Capacity/thermal/power measurements.
ADR-0006	Wi-Fi vs cellular egress design	TEST-NET-EGRESS-001.
ADR-0007	MDM provider / enrollment method	ABM eligibility, supervised feature needs, API/export/cost.
ADR-0008	Backend stack	Team skills, Windows-node integration, deployment target.
ADR-0009	Object storage and media retention	Creator authorization/privacy/backup needs.
ADR-0010	Android pilot yes/no	Actual platform need vs engineering convenience.
ADR-0011	Proxy/VPN use if any	Documented security/business requirement; provider terms.
ADR-0012	Production update strategy	Staging pod, OS app compatibility, vendor controller updates.

ADR template

ADR-XXXX - <title>

Status: Proposed | Accepted | Rejected | Superseded

Date: YYYY-MM-DD

Owners: ...

Context

What decision is required and why now?

Evidence

- Internal observations/test IDs
- Primary-source links
- Vendor facts
- Assumptions explicitly labeled

Options

A ...

B ...

C ...

Decision

Chosen option and exact scope.

Consequences

Benefits, costs, risks, operational load.

Rollback / exit

How to reverse without losing fleet state.

Follow-up

Tests, procurement, code changes, review date.

20. Claude execution backlog - first 30 work items

ID	Work item	Acceptance evidence
PF-001	Interview Dim and capture tested controller evidence	docs/source-ledger.md updated; exact product/version/hardware known.
PF-002	Inventory existing 20-device enclosure	Photos, PSU/fan/port specs, slot map.
PF-003	Inventory existing phones	CSV/database import with model/OS/battery/ownership state.
PF-004	Create unknowns ledger	Every \$8/\$11 internal-brief unknown mapped to owner/test.
PF-005	Create ADR-0001/0002	Architecture/safety baseline accepted.
PF-006	Create minimal UIKit input test app	Logs touch type with build/version evidence.

ID	Work item	Acceptance evidence
PF-007	Run TEST-IOS-INPUT-001	Report and artifacts complete.
PF-008	Build isolated two-device control WLAN	No public exposure; diagrams and ACLs documented.
PF-009	Run TEST-NET-EGRESS-001	Route behavior documented.
PF-010	Install chosen iOS controller on dedicated Windows host	Hash/version/services/ports captured.
PF-011	Run controller security review	Firewall/AV exception decision documented.
PF-012	Run 2-device mirror/control soak	Reconnect/resource metrics collected.
PF-013	Run media import test	Format/latency/failure behavior documented.
PF-014	Create backend repository and CI	Lint/test/migrations pipeline green.
PF-015	Implement PostgreSQL schema	Migration + constraints + test fixtures.
PF-016	Implement auth/RBAC skeleton	Roles and creator scoping tests.
PF-017	Implement node registration/mTLS plan	Unique node identity and revocation path.
PF-018	Implement heartbeat endpoint	Dashboard can show stale/healthy nodes.
PF-019	Implement iMouse adapter health/list	No arbitrary macro endpoint.
PF-020	Implement approved media import task	Hash checked, audited, idempotent.
PF-021	Implement diagnostic screenshot task	Retention and reason captured.
PF-022	Implement human session lock	One operator/device at a time, outcome logged.
PF-023	Build device/rack dashboard	Physical slot and assignment visible.
PF-024	Build asset approval model	Unapproved media cannot be queued.
PF-025	Build account authorization model	Assignment requires authorization_ref.
PF-026	Assemble five-device pod	Known BOM/wiring labels.
PF-027	Run five-device power/thermal/network soak	Measured capacity sheet produced.
PF-028	Run failure drills	Node agent restart, controller restart, DB/Redis outage behavior.
PF-029	Review current platform rules	Source ledger dated before pilot operations.
PF-030	Gate-2 review	Written go/no-go for 10-device expansion.

21. Claude startup prompt embedded in the handoff

Use this as the opening instruction when handing over the PDF

The prompt below is intentionally explicit so the coding model does not skip the evidence phase or reinterpret operator anecdotes as implementation requirements.

You are the technical lead for XYZ MGMT's physical device-fleet project.

Treat Part A of this PDF as the current architecture authority and Part B as the deep engineering reference.

Before writing production code:

1. Read the READ THIS FIRST section and the verification ledger.
2. Create an Unknowns Ledger and ADRs.
3. Obtain the missing hands-on facts from Dim; do not guess the name/version of his tested control product or what hardware is already owned.
4. Close the blocking input-type, egress, controller-security and media-import tests on a two-device bench.
5. Build the durable 5-device pilot before any 20+ scale decision.

Do not implement or optimize: bulk account creation, bulk friend/follow/add behavior, fake engagement, scraped-user acquisition, fingerprint spoofing, location falsification, automatic ban replacement, CAPTCHA/rate-limit bypass, or a generic third-party social-UI macro engine.

Use a typed control plane: FastAPI + PostgreSQL + Redis + a Windows node agent + vendor adapters. Keep the vendor iOS controller local and isolated. Use MDM for governance and the local controller for human remote operation/approved media handling.

For every task you propose, include: evidence/source, dependencies, exact output artifact, acceptance criteria, rollback, and which ADR/test it closes.

If evidence conflicts, separate observation from interpretation and flag the conflict rather than silently reconciling it.

22. Web verification ledger - research cut 26 August 2026

The following sources were re-checked for this master handoff. First-party sources are preferred for operating-system behavior and platform rules. Vendor sources establish what a vendor claims its product supports; they do not establish platform invisibility, security or production stability.

ID	Source / URL	Quality / use
P01	Snapchat Support - account locked / SS06 SS18 SS07 https://help.snapchat.com/hc/en-us/articles/10215197231892-My-account-was-locked-for-violating-Snap-s-Community-Guidelines	First-party - Device bans; SS07 too many accounts associated; banned device cannot create new accounts.
P02	Snapchat Support - Why does it say my device is banned? https://help.snapchat.com/hc/en-us/articles/48008135555860-Why-does-it-say-my-device-is-banned	First-party - Ban applies to phone/tablet; can affect used/refurbished devices.
P03	Snap Sexual Content - March 2026 https://values.snap.com/policy/policy-community-guidelines/sexual-content	First-party - Prohibits offers/solicitation of online sexual subscription services.
P04	Apple DeviceCheck documentation https://developer.apple.com/documentation/devicecheck	First-party - Two per-device bits and server API.
P05	Apple WWDC21 - Mitigate fraud with App Attest and DeviceCheck https://developer.apple.com/videos/play/wwdc2021/10244/	First-party - DeviceCheck state persists across reinstall, transfer, Erase All Contents and Settings.
P06	Android - Best practices for unique identifiers https://developer.android.com/identity/user-data-ids	First-party - Restrictions on IMEI/serial/MAC; favor resettable identifiers.
P07	Android Play Integrity - Device recall (beta) https://developer.android.com/google/play/integrity/device-recall	First-party - Three custom per-device values can persist after reinstall/reset.
P08	Apple UIApplicationSupportsIndirectInputEvents https://developer.apple.com/documentation/bundle-resources/information-property-list/uiapplicationsupportsindirectinputevents	First-party - indirectPointer vs direct compatibility behavior; iOS 17+ default.
P09	Apple iOS/iPadOS 17 release notes https://developer.apple.com/documentation/ios-ipados-release-notes/ios-ipados-17-release-notes	First-party - SDK/default change for indirect input.
P10	Apple Pointer interactions https://developer.apple.com/documentation/uikit/poininter-interactions	First-party - Apps can distinguish pointing-device touch events.
P11	Apple VPN overview for deployment https://support.apple.com/guide/deployment/depae3d361d0/web	First-party - Per-app VPN and Always On VPN behavior.
P12	Apple IKEv2 management settings https://support.apple.com/guide/deployment/ikev2-settings-dep4ce9487d/web	First-party - Always On VPN can configure cellular/Wi-Fi.
P13	Snapchat App Store https://apps.apple.com/us/app/snapchat/id447188370	First-party marketplace - Current listing requires iOS 15.0+.
P14	Instagram App Store https://apps.apple.com/us/app/instagram/id389801252	First-party marketplace - Current listing requires iOS 16.3+.
P15	TikTok App Store https://apps.apple.com/us/app/tiktok-videos-shop-live/id835599320	First-party marketplace - Current listing requires iOS 14.0+.
P16	Apple - iOS 26 compatible iPhone models https://support.apple.com/guide/iphone/iph3fa5df43/ios	First-party - Current iOS 26.5 supported models include iPhone 11 and SE2+.
P17	Apple iOS 16 compatible devices https://support.apple.com/en-la/103267	First-party - iPhone 8/X and newer; excludes 6s/7.
P18	Apple acceptable operating temperatures https://support.apple.com/en-mide/118431	First-party - 0-35 C ambient operating range; heat can shorten battery life.
P19	Apple thermally limited charging https://support.apple.com/en-ie/guide/iphone/iph3006fbee4/ios	First-party - Charging can slow/pause when temperature is abnormal.
P20	Apple Platform Deployment - July 2026 https://support.apple.com/guide/deployment/welco me/web	First-party - Current organization deployment reference.
P21	Apple Automated Device Enrollment https://support.apple.com/guide/security/automate	First-party - Automatic MDM enrollment and supervision.

ID	Source / URL	Quality / use
	d-device-enrollment-secc2cd563ef/web	
P22	Apple device supervision https://support.apple.com/guide/deployment/about-device-supervision-dep1d89f0bff/1/web/1.0	First-party - Supervision for organization-owned devices.
P23	Apple managed app distribution https://support.apple.com/en-ie/guide/deployment/dep575bfed86/web	First-party - Managed apps can be assigned network relay/per-app VPN.
P24	Android dedicated devices https://developer.android.com/work/dpc/dedicated-devices	First-party - Fully managed dedicated device pattern.
P25	Android Management API provisioning https://developers.google.com/android/management/provision-device	First-party - Enrollment tokens, fully managed/dedicated modes.
P26	scrcpy official GitHub https://github.com/Genymobile/scrcpy	Project primary - Android USB/TCP mirroring/control; no root; cross-platform.
P27	Android Debug Bridge https://developer.android.com/tools/adb	First-party - Authorized Android debug/device bridge.
P28	TikTok Community Guidelines https://www.tiktok.com/community-guidelines	First-party - Deceptive behavior/fake engagement and account enforcement.
P29	TikTok - How TikTok counters deceptive behaviour https://newsroom.tiktok.com/how-tiktok-counters-deceptive-behaviour	First-party - States automation to register/operate accounts in bulk is not allowed.
P30	TikTok Accounts and Features guidelines https://www.tiktok.com/community-guidelines/en/accounts-features	First-party - Enforcement and attempts to avoid enforcement.
P31	TikTok Developer Guidelines https://developers.tiktok.com/docs/en/our-guidelines-developer-guidelines	First-party - Approved integrations/app review.
P32	Reddit Rules https://redditinc.com/policies/reddit-rules	First-party - Authentic participation; no spam/content manipulation.
P33	Reddit User Agreement - July 2026 https://redditinc.com/policies/user-agreement	First-party - Limits disruptive/overburdening automated use.
P34	Reddit Developer Terms https://redditinc.com/policies/developer-terms	First-party - Commercial data/API restrictions.
V01	Some3C iPhone Farm Settings https://doc.some3c.com/iphone-farm-setup/iphone-farm-settings	Vendor - Same network requirement; Windows software setup; security-exclusion recommendation.
V02	Some3C iMouse API documentation https://doc.some3c.com/iphone-farm-setup/api-documentation	Vendor - HTTP/WebSocket, device management, media, OCR/image/color capabilities.
V03	Some3C iMouse XP documentation https://doc.some3c.com/iphone-farm-setup/imouse-xp-new-version	Vendor - Windows 10+ and vendor installation requirements.
V04	iMouse product overview https://www.imouse.cc/	Vendor - Hardware + AirPlay model, HTTP/WebSocket APIs, iOS support claims.
V05	Some3C configuration API https://doc.some3c.com/xp-api-documentation/configuration-related	Vendor - Local API examples, ports, allow_ip_list, frame settings.
V06	Phones.farm iMouse product page https://phones.farm/products/imouse-iphone-jailbreak-free-central-control-ios-automation-otg-game-studio-hardware-synchronizer-screencast-control-board	Vendor/store - Observed retail price snapshot for control board.

23. Internal research digest - facts Claude must preserve

This section compresses the uploaded v1.1 brief into architecture-relevant facts without converting the described high-risk operator tactics into an implementation recipe. Consult the original internal markdown for verbatim operator quotes if needed.

Topic	Internal research signal	How to use it
Two build concepts	A: 100-300 Snapchat disposable devices; B: 5-30 durable Instagram/TikTok devices.	Keep separate in analysis; do not blend their economics/assumptions.
Dim	Has tested a Chinese control/mirroring product and owns practical failure knowledge.	Primary interview before purchases/coding.
Operator A	Cheap iPhone 6s, proxy/device, manual-heavy, ~14-20 day account life, reported traffic/revenue.	Business hypothesis only; yield claims unverified; enforcement cycling not

Topic	Internal research signal	How to use it
		implementation scope.
Operator B	Hundreds of iPhones, iMouse/OTG HID, browser dashboard, OCR/vision, carrier data, cloud media queue, one employee.	Architecture evidence strengthened by vendor docs; Snapchat-specific conclusions remain weak because he had not scaled Snapchat.
Convergence	Both operators saw value in funnelling toward durable assets rather than pure burn methods.	Supports durable-pod priority, independently of compliance concerns.
Open #1	Actual iMouse pointer event semantics.	Resolved only by owned test app.
Open #2	Actual SIM vs Wi-Fi egress while mirroring.	Resolve by controlled network test.
Open #3	Android reset-surviving identity/state.	Partially resolved: Play Integrity device recall proves persistent backend state is possible.
Chassis	At least one 20-device enclosure already exists internally.	Inspect before specifying replacement.
Internal n=1 test	Traffic increased after fifth public-profile story; 147 then 182 clicks and 19 then 29 free-page subscribers reported.	Hypothesis H-SNAP-01; replicate before treating as threshold.
Economics	200-device iOS/Android models and labour estimates are modeled, not observed.	Keep assumptions editable; replace with pod-level measured TCO.

23.1 Unresolved questions that remain after this master update

- What exact control/mirroring product did Dim test, what version, on how many devices, for how long, and what failed?
- What exact chassis and power components are already owned?
- Does the exact iMouse HID/AssistiveTouch path produce indirectPointer in the test app on current iOS?
- Can local AirPlay/control coexist with stable cellular Internet egress in the desired topology?
- What is measured mirror bitrate/resource cost per device at the chosen resolution/fps?
- What is the actual per-phone monthly data consumption for the intended compliant content workflow?
- What is the reliable content-import path and its throughput on current iOS?
- Which MDM provider will be used and can the device supply chain be added to Apple Business Manager / supervised appropriately?
- What is the legal/contractual status of agency access to each creator/platform account in the intended jurisdictions?
- What conversion/revenue metric actually justifies a phone-required workflow versus official schedulers/APIs/paid acquisition?

24. Final handoff checklist

Claude should be able to answer these before recommending scale

If any answer is "unknown," the output should point to the specific test, owner or source needed - not guess.

- ☐ What hardware and software does XYZ already own?
- ☐ Which controller is selected, and why is it better than the alternative actually tested by Dim?
- ☐ What are the controller's listening ports, trust boundaries and endpoint-security requirements?
- ☐ What event type does the controller produce in the owned iOS input test app?
- ☐ What interface carries Internet traffic under the selected control topology?
- ☐ What is the measured devices-per-node capacity at target mirror quality?
- ☐ What is the full-load power draw and thermal profile of one pod?
- ☐ What iPhone model floor is being purchased and what current OS/app-support evidence supports it?
- ☐ How are devices supervised/managed and wiped/retired?
- ☐ How are approved media assets imported without losing attribution/audit information?

- ☐ How does a human operator take exclusive control of one device and report the outcome?
- ☐ Which actions are explicitly blocked from automation?
- ☐ How are credentials, media and creator boundaries protected?
- ☐ How is a platform warning/enforcement event handled without automated circumvention?
- ☐ What are the 12-month pod TCO and measured operator minutes per useful outcome?
- ☐ What evidence is required before 5 -> 10, 10 -> 20, and 20 -> multi-pod expansion?

Part B follows

Engineering reference guide

The pages after this handoff are the original 91-page OFM Phone Farm Engineering Guide. Use it for deeper chapters on racks, power, Wi-Fi, VLANs, NAT/CGNAT, DNS, proxies, VPNs, Apple/Android fleet management, control-plane design, media workflows, analytics, security, incident response, BOMs, schemas and runbooks. Where Part A conflicts with an earlier assumption in Part B, Part A is the current decision layer.

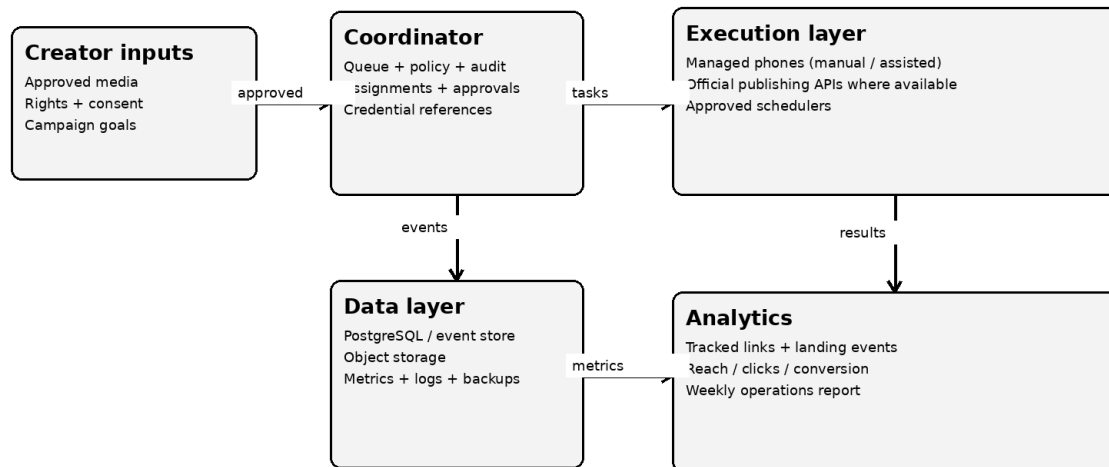
OFM PHONE FARM ENGINEERING GUIDE

Physical infrastructure, managed-device fleets, networking, software control planes, publishing workflows, analytics, security, reliability and operational governance

Research edition - 26 August 2026

Reference Architecture

A compliant, observable creator-traffic operations platform



Reference architecture used throughout this guide.

Scope: legitimate creator-traffic operations using organization-owned devices, official management tooling, authorized accounts, approved publishing paths and auditable employee workflows.

Reader notice and safety boundary

This manual explains phone-farm engineering as a fleet-operations problem. It covers hardware, power, thermal design, USB, Wi-Fi, cellular concepts, NAT, CGNAT, DNS, VPNs, forward proxies, MDM, Android Enterprise, ADB, device/coordinator communications, software architecture, content operations, analytics, security, staffing, incident response and scaling. It is written for an OFM or creator-management environment, but most engineering material also applies to QA labs, retail kiosks, field-device fleets and social-media production teams.

What is intentionally not operationalized

The guide does not provide recipes for defeating platform enforcement, concealing a device or account fingerprint, falsifying geographic location, rotating identities to evade bans, automating account creation at scale, manufacturing fake engagement, bypassing rate limits, defeating CAPTCHAs, or scripting third-party social UIs to imitate humans where the platform prohibits automation. Those techniques create platform, fraud, security and contractual risk. Proxy/VPN and device-identity topics are therefore explained for networking, privacy, segmentation, testing and observability - not evasion.

Some OFM-specific claims circulating in operator communities are anecdotal. They are labeled as such in this guide. Technical claims about fleet management, device provisioning, power, temperature, publishing APIs and platform rules are anchored primarily to first-party documentation listed in the References section. Platform rules change; re-check them before deployment.

How to use this guide

If you are...	Start with	Then read
Owner / agency principal	Ch. 1-5, 55-61	Costs, risks, operating model, roadmap
Infrastructure engineer	Ch. 6-23	Power, rack, Wi-Fi, routing, VPN/proxy/DNS
Mobile/fleet engineer	Ch. 24-31	Apple MDM, Android Enterprise, ADB, enrollment
Backend engineer	Ch. 32-41	Coordinator, queues, APIs, database, telemetry
Operations manager	Ch. 42-54	Content pipeline, employees, analytics, SOPs
Security/compliance	Ch. 4, 52-59	Access, logs, incidents, privacy, offboarding
Builder starting a pilot	Ch. 60-65 + App. A-I	BOM, acceptance tests, build sequence

Contents

PART I - Purpose, industry model and boundaries

1. Phone farms in OFM
2. Traffic funnel and business model
3. What physical phones solve - and what they do not
4. Platform-policy and authorization boundary
5. Architecture decision: API-first, device-first or hybrid

PART II - Physical and electrical engineering

6. Device selection
7. Rack and mechanical design
8. Power architecture
9. USB topology and cabling
10. Thermal engineering
11. Fire/electrical safety and UPS
12. Host computers and local controllers
13. Inventory, labeling, spares and lifecycle

PART III - Networking, connectivity, proxies and tunnels

14. Network reference architecture
15. Wi-Fi radio and capacity planning
16. Ethernet, switches and VLANs
17. IPv4, IPv6, NAT and CGNAT
18. DNS and resolver design
19. Forward proxies and proxy protocols
20. VPNs, tunnels and per-app routing
21. Cellular, SIM and eSIM concepts
22. Egress observability and troubleshooting
23. Network failure modes

PART IV - Device fleet management

24. Apple fleet architecture
25. Supervision, enrollment and MDM
26. Android Enterprise architecture
27. Android provisioning and policy
28. ADB for authorized engineering
29. App distribution and configuration
30. OS updates and maintenance rings
31. Health telemetry and remote actions

PART V - Software control plane

32. Control-plane architecture
33. Device-to-coordinator communications
34. Task queue and state machine
35. Database and entity model
36. Object/media storage
37. API design
38. Messaging and event transport
39. Metrics, logs and traces
40. Credential and secret handling
41. Availability, backup and disaster recovery

PART VI - OFM operations, content and analytics

42. Creator onboarding and authorization
43. Media ingest and rights metadata
44. Content processing and variants
45. Approval workflow
46. Publishing paths and official APIs
47. TikTok operational notes
48. Reddit, X and Snapchat operational notes
49. Instagram/Meta operational notes
50. OnlyFans boundary and account responsibility

- 51. Tracking links and attribution
- 52. Employee roles and access model
- 53. Daily/weekly operating cadence
- 54. Quality control and audit

PART VII - Security, compliance and reliability

- 55. Threat model
- 56. Platform restriction and enforcement handling
- 57. Incident response
- 58. Privacy and data minimization
- 59. Business continuity and creator exit

PART VIII - Scaling and implementation

- 60. Scaling from 5 to 100+ devices
- 61. Cost model and BOM categories
- 62. Pilot build - 10 device reference
- 63. 25-50 device expansion
- 64. 30/60/90 day implementation roadmap
- 65. Acceptance test plan

APPENDICES

- A. Example IP/VLAN plan
- B. Example firewall policy
- C. Task JSON and API examples
- D. Example SQL data model
- E. MQTT topic design
- F. Device commissioning checklist
- G. Weekly operations report template
- H. Incident runbooks
- I. Capacity formulas
- J. Glossary
- K. Research references

PART I - PURPOSE, INDUSTRY MODEL AND BOUNDARIES

A phone farm is only useful when the business process around it is explicit. This part defines the OFM use case, separates legitimate fleet operations from manipulation, and establishes the architecture used throughout the manual.

1. Phone farms in OFM

In an OFM context, a phone farm is a managed collection of physical mobile devices used by an agency or creator team to operate authorized social accounts, prepare and publish content, review notifications, collect performance signals and support traffic acquisition. The physical phones are execution endpoints; the real system is the combination of devices, people, network, content rights, credentials, workflow software and analytics.

Industry usage and terminology

Public operator communities use “phone farm” loosely. It can mean ten old iPhones on a desk, a rack of Android phones, a motherboard rack without screens, a set of remotely controlled devices, or a larger operation with coordinators and staff. These reports are anecdotal and should not be treated as evidence that a particular platform will tolerate the associated behavior.

The legitimate engineering interpretation is closer to a corporate mobile fleet or device lab: each device is inventoried, assigned, secured, connected to controlled networks, monitored and maintained. The difference is that the workload is creator-content operations rather than QA testing or retail kiosk use.

- Define the owner of every device and social account.
- Treat creator authorization and content rights as first-class records.
- Separate device administration from social publishing.
- Measure output with business metrics rather than “number of phones.”

What the farm is not

A physical phone is not an invisibility device. Platforms can evaluate account behavior, content similarity, network characteristics, login history and many other signals. A real handset may make the workflow more operationally convenient, but it does not convert prohibited behavior into permitted behavior.

The architecture should therefore optimize for repeatability, security, operator throughput, reliability and attribution. It should not optimize for hiding the existence of the fleet.

- Do not buy hardware before mapping the traffic funnel.
- Do not assume one physical device must equal one account. The correct mapping depends on authorization, platform rules, staffing and operational risk.

Engineering checklist

- Write down the business goal for each platform.
- List authorized creator accounts and account owners.
- Define which actions are manual, API-driven or scheduler-driven.
- Create a platform-rule register with review dates.

2. Traffic funnel and business model

The phone-farm layer sits in the middle of a larger creator acquisition funnel. Engineering decisions should be evaluated by their effect on measurable funnel stages rather than raw posting volume.

Reference funnel

A typical funnel is: approved creative asset -> platform post -> profile visit -> tracked link or landing page -> creator page -> subscription or other paid action. Depending on the business, there may be additional layers such as a link hub, geofenced landing page, email capture, or CRM attribution. Every added layer creates potential conversion loss and additional privacy/security obligations.

The control plane should attach campaign, creator, platform, account, device, operator and asset identifiers to each publishing task. The analytics plane should connect those identifiers to post-level reach where available and to downstream click/conversion events where lawful and technically possible.

- Primary operational metrics: task completion rate, post success rate, account availability, content approval latency.
- Primary marketing metrics: reach, profile visits, click-through rate, landing conversion, subscriber conversion, revenue per thousand impressions where calculable.
- Primary reliability metrics: device heartbeat, battery/thermal status, network packet loss, API error rate, queue age.

Unit economics

A useful model treats each account or campaign as a cost center. Hardware amortization, software, connectivity, employee time, content production and creator revenue share should be visible. "More phones" only creates value if incremental gross contribution exceeds incremental infrastructure and labor cost.

Attribution is imperfect because not every platform exposes the same analytics and some downstream conversions cannot be deterministically tied to a post. The system should distinguish observed events from modeled or inferred attribution.

Layer	Example measure	Engineering owner
Creative	Approved assets per week; QC rejection rate	Content ops
Publishing	Task success; queue age; retries	Fleet/backend
Platform	Reach; profile actions	Growth ops
Landing	Unique clicks; conversion	Web/backend
Revenue	Paid conversion; ARPU/LTV	Finance/analytics

3. What physical phones solve - and what they do not

Physical devices are valuable when the workflow actually requires the native mobile app, hardware-bound capabilities, mobile camera/media access, or a standardized operator interface. They are less valuable when an official API or web console already provides the required function.

Control Plane vs Data Plane

Do not mix fleet administration with creator content and social traffic

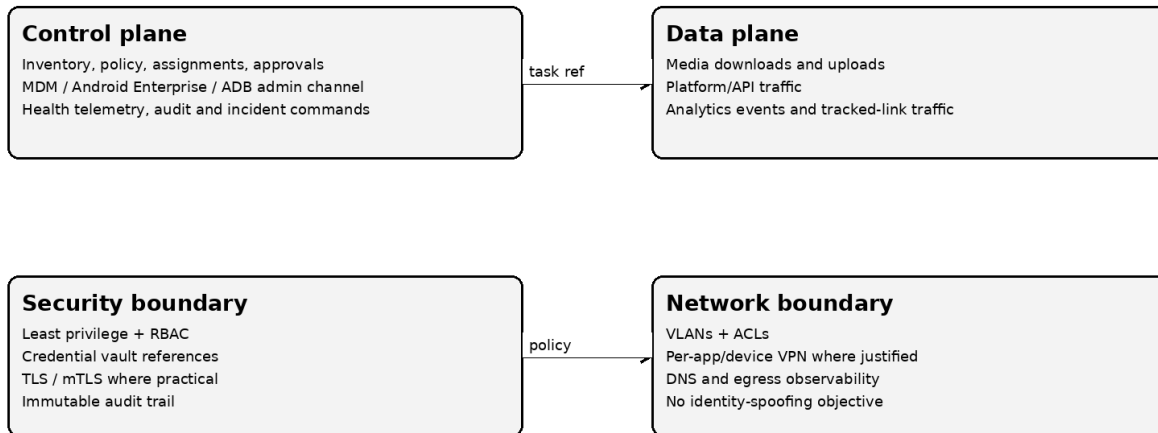


Figure 3-1. Control-plane and data-plane separation.

Problems physical devices solve

Phones provide the authentic application runtime, camera roll, push notifications, biometric-secured credential storage, mobile-only creation tools and native moderation/status screens. Organization-owned devices can also be physically segmented from employees' personal phones.

For Android, corporate devices can be fully managed or dedicated under Android Enterprise. For Apple fleets, organization-owned supervised devices can receive stronger MDM controls than unsupervised personal devices. [S1][S5]

- Native-app-only features.
- Consistent operator work surface.
- Separation between personal and corporate credentials.
- Physical inventory that can be wiped, reassigned and retired.

Problems they do not solve

Phones do not remove platform terms, content restrictions, API rules or behavioral enforcement. They also do not automatically solve account ownership, credential leakage, employee mistakes, attribution or creator-consent management.

A rack can even make risk worse if the same content is repetitively distributed, credentials are broadly shared, devices are not patched, or network architecture creates one large blast radius.

- Build authorization and access control before scaling hardware.
- Use official publishing APIs where available and suitable.
- Keep humans in approval loops for ambiguous or high-risk actions.

4. Platform-policy and authorization boundary

The most important non-hardware requirement is an explicit rule boundary. Different platforms permit multiple accounts, third-party management, APIs and scheduling to different degrees, but commonly prohibit spam, fake engagement, coordinated inauthentic activity, automated bulk account operation, or circumvention of enforcement. These rules can change.

Policy update rule

Treat all platform-specific operating rules in this manual as a dated snapshot, not a permanent guarantee. Re-check first-party policies and API terms before deploying a workflow change.

Current examples from first-party rules

TikTok's integrity rules distinguish authentic multi-account use from deceptive or spam behavior and prohibit automation used to operate accounts in bulk or manipulate platform systems. TikTok also provides a formal Content Posting API for authorized applications, with scopes and audit requirements. [S22][S23]

Reddit's rules require authentic participation and prohibit spam and manipulation. Snapchat explicitly prohibits spam and artificially inflated engagement. X permits legitimate third-party account management but prohibits coordinated inauthentic activity, duplicate amplification and ban-evasion behavior. [S24][S26][S27]

Authorization records

For every creator and account, retain a signed or otherwise auditable authorization record stating who owns the account, what the agency may do, which employees or systems may access it, what content is approved, and what happens on termination. The agency should not rely on informal chat messages as its only control evidence.

Keep platform credentials separate from legal identity documents and sensitive creator files. The system should store credential references to a vault rather than passwords in application tables or spreadsheets.

- Creator ID and contract/authorization reference.
- Account owner and recovery email/phone owner.
- Permitted actions and publishing platforms.
- Content usage scope and expiration/revocation process.
- Offboarding/wipe/recovery procedure.

5. Architecture decision: API-first, device-first or hybrid

The correct architecture depends on which functions are officially exposed by platform APIs and which require a person in a native application. A hybrid design is usually the most maintainable: APIs or approved schedulers handle supported publishing and analytics; physical phones handle native-only tasks and human review.

Reference Architecture

A compliant, observable creator-traffic operations platform

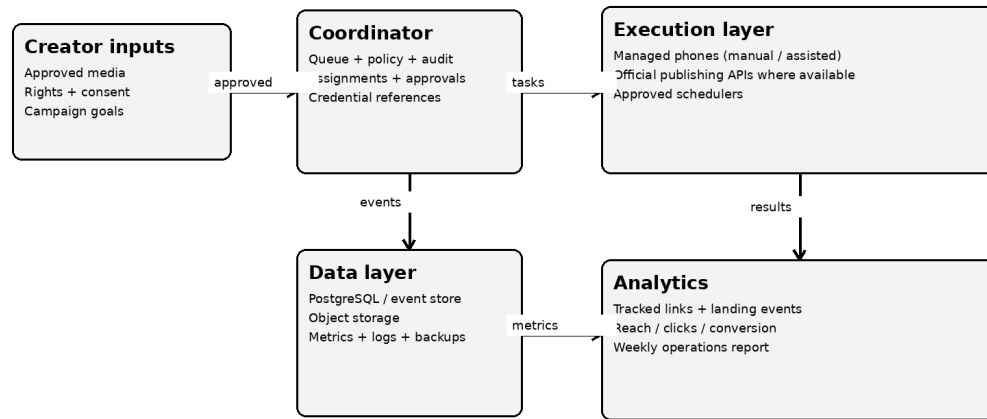


Figure 5-1. Hybrid reference architecture.

API-first

Use an API-first workflow when the platform provides a supported publishing or analytics endpoint and the agency can meet app-registration, scope and audit requirements. API-first systems are easier to log, retry, rate-limit, test and attribute. TikTok's Content Posting API is one example of an explicit, authorized posting path. [S22]

- Advantages: deterministic requests, auditability, fewer physical touch points, easier scaling.
- Costs: developer integration, platform app review, scope limitations, changing APIs.

Device-first

Device-first means a human operator performs most social actions on managed phones. This is slower to scale but can be appropriate for native-only workflows. Fleet software should coordinate assignments and record outcomes without pretending to be the human.

If the platform does not permit automated UI operation, do not turn accessibility tools, ADB input commands or computer vision into a covert substitute for an official API.

- Advantages: access to native UX; human judgment.
- Costs: labor, training, ergonomic complexity, more devices, weaker deterministic automation.

Hybrid

Hybrid systems place a coordinator above both paths. A task can route to an official API executor, an approved scheduler, or a human/device queue depending on platform, account and action type. All paths write back a common result schema.

Dimension	API-first	Device-first	Hybrid
Scale	High where API permits	Labor-bound	High for supported actions
Auditability	Strong	Requires operator capture	Strong if unified
Native-only features	Limited	Strong	Strong
Platform dependence	API contracts	App UX changes	Both
Recommended role	Default where supported	Exception/native tasks	Overall architecture

PART II - PHYSICAL AND ELECTRICAL ENGINEERING

A rack of phones is a continuous electrical and thermal load, not a collection of casual chargers. This part treats the installation as small infrastructure: load planning, serviceability, airflow, cable discipline and lifecycle management.

6. Device selection

Device choice should be driven by supported OS life, battery condition, application compatibility, management support, reparability and total cost rather than the cheapest used handset price.

Selection criteria

Prefer models that remain inside the OS/security support window for the expected service life. Confirm the required social apps still support the OS version. For Android, verify Android Enterprise management capabilities and consider zero-touch eligible procurement for larger deployments. For Apple, organization-owned devices that can be enrolled and supervised through the intended deployment path are easier to administer consistently. [S1][S5]

Used devices can lower capital cost but introduce unknown battery health, replaced parts, activation locks, inconsistent storage and shorter remaining update life. A receiving inspection process is therefore mandatory.

- Supported OS/security updates for planned life.
- 64 GB or more storage if local media caching is heavy; size to actual workflow.
- Healthy battery and charging port.
- Stable Wi-Fi radios and compatible bands.
- Consistent model families to reduce spare-parts and cable variety.
- No activation lock, unknown MDM enrollment or carrier financing lock.

Screened vs headless hardware

Phone-motherboard racks and screenless devices appear in operator communities, but they add maintenance and compliance uncertainty because the hardware may be modified or difficult to recover. For a professional pilot, normal complete phones are easier to inventory, reset, update, visually inspect and hand to an operator.

Only consider custom hardware after the software architecture and workload are proven on standard devices.

Factor	Pilot preference	Scale preference
Model mix	1-2 models max	Standardize by cohort
OS age	Current or recent	Long remaining support
Battery	Known healthy	Replaceable service process
Procurement	Reputable used/new	Enterprise/reseller where possible
Management	Manual + MDM/Enterprise	Automated enrollment

7. Rack and mechanical design

The mechanical objective is not maximum density. It is stable airflow, easy identification, fast replacement, safe cabling and enough access that an operator can service one phone without disturbing ten others.

Mechanical Rack Concept

Serviceable layout: airflow, labeling, power distribution and cable relief

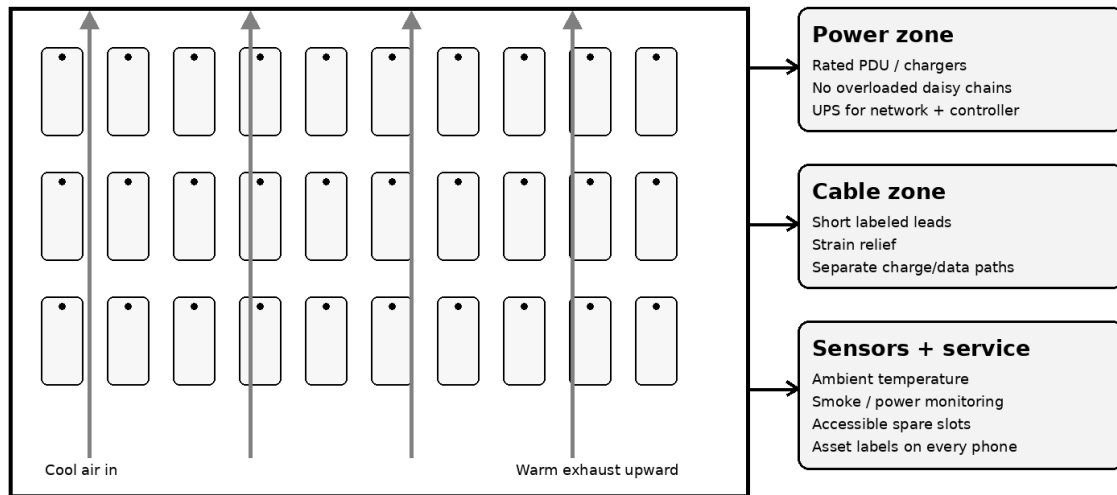


Figure 7-1. Example serviceable rack layout.

Slot geometry

Use vertical or slightly tilted slots so screens remain visible and warm air can rise between devices. Leave air gaps instead of clamping phones tightly together. The required spacing depends on device power, ambient temperature and fan airflow; validate it with temperature measurements rather than a fixed internet rule.

Every slot should have a stable slot ID. The physical label, inventory database and cable label should use the same identifier. Avoid relying on “third phone from the left.”

- Front access to power button and charging connector.
- No sharp edges or pressure on battery area.
- Cable bend radius and strain relief.
- Fan guards and removable dust filters where appropriate.
- A service aisle or pull-out tray for dense racks.

Material choices

Nonconductive trays simplify electrical isolation. Metal enclosures can provide stronger structure but require proper grounding and careful cable protection. 3D-printed fixtures should be made from suitable materials and kept away from hot power components; do not assume hobby plastics are fire-rated.

Keep chargers, AC distribution and high-current DC distribution in a power zone separate from phone slots. This improves cooling and makes fault isolation easier.

Engineering checklist

- Assign one immutable slot ID to every physical position.
- Provide a safe way to remove one device without pulling adjacent cables.
- Measure inlet and exhaust temperature under full load.
- Document rack weight, power-feed rating and emergency disconnect location.

8. Power architecture

Power should be designed from measured load. Charging power is dynamic: a phone may draw more while recovering from low battery, then taper as it approaches full charge, and thermal controls can reduce charging rate. USB Power Delivery negotiates power between compatible source and sink; USB PD 3.1 supports substantially higher power levels than phones normally require, so charger nameplate wattage is not the same as actual phone draw. [S10]

Illustrative Power Scaling

Planning example only - not a specification for any particular phone model

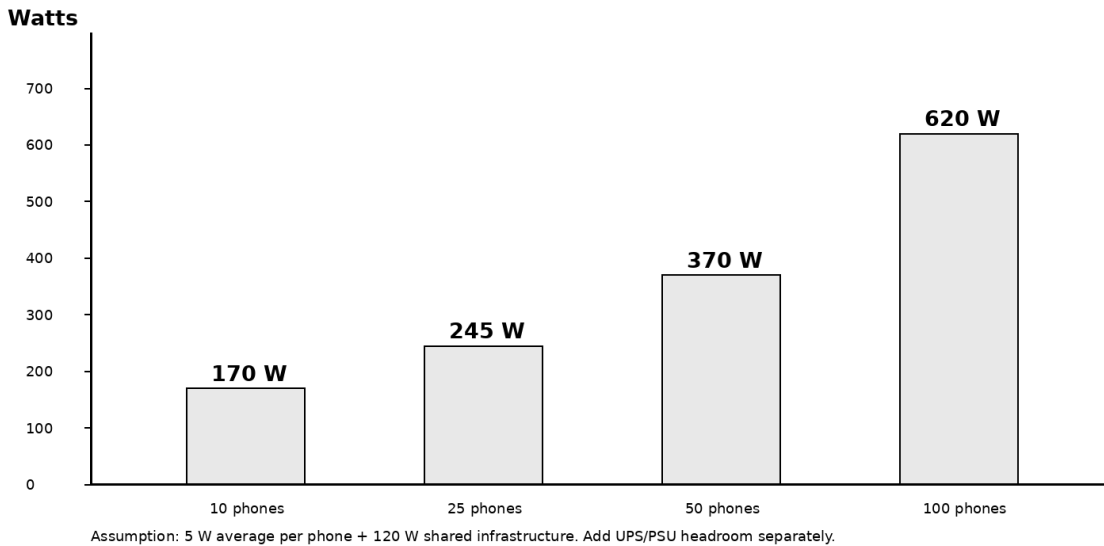


Figure 8-1. Illustrative power scaling model; use measured values for design.

Load model

For planning, define P_{devices} as the sum of measured average phone power, P_{peak} as the likely coincident peak, and P_{infra} as routers, switches, APs, controller PCs, fans and monitoring. Select supplies, branch circuits and UPS capacity with engineering headroom rather than operating continuously at nameplate limits.

Illustrative example only: if 25 phones average 5 W while connected, device load is about 125 W. Add the actual measured infrastructure load and then add headroom. This example is not a specification for any handset.

- Measure at the AC input with a power meter during representative use.
- Record both idle/maintenance load and recovery-from-low-battery load.
- Do not assume all multi-port chargers can deliver their advertised maximum to every port simultaneously.
- Use chargers/cables from reputable vendors with appropriate certifications for your region.

Power zoning

Split large racks into independently protected zones. A single failed adapter or tripped protection device should not remove the whole fleet. Network, control server and essential monitoring may deserve UPS runtime even if phone charging itself is not fully UPS-backed.

If chargers expose smart power allocation, test how port behavior changes when many devices are connected or disconnected at once.

Engineering checklist

- Create a port-to-slot map.
- Label every power supply with zone and rated output.
- Avoid improvised high-current wiring.
- Document shutdown/startup sequence after utility failure.

9. USB topology and cabling

USB can carry power, data or both. A rack that uses ADB over USB or Apple configuration tooling has different requirements from a charge-only rack. Treat the USB data topology as an engineered bus, not an unlimited fan-out.

Data vs charge

Charge-only cables reduce accidental data exposure but cannot support local USB administration. Data-capable cables are required for USB ADB and some provisioning workflows. Mark cable type visibly because identical connectors can have different capabilities.

Powered USB hubs should have their own adequate supplies. Avoid long chains of hubs; every tier adds connectors, failure points and potential bandwidth or enumeration problems. For large fleets, distribute devices across multiple host USB controllers and powered hubs rather than one massive tree.

- Use short, labeled cables.
- Maintain a spare cable bin by exact type.
- Log recurring disconnects by slot; intermittent cables cause disproportionate troubleshooting time.
- Use strain relief so the phone connector is not supporting cable weight.

Host enumeration

For Android, ADB identifies attached authorized devices by serial and transport. For Apple, management is normally handled through MDM for ongoing fleet operations, with Configurator or local tooling used for specific provisioning/service cases. [S3] [S5]

Symptom	Likely class	Check first
Phone charges, no data	Charge-only cable / port mode	Known-good data cable
Random disconnects	Cable/hub/power instability	Hub supply + logs
Many devices disappear together	Shared hub/controller fault	Upstream hub and AC zone
Slow media transfer	Shared USB bandwidth	Controller topology

10. Thermal engineering

Heat is a reliability and battery-lifetime problem. Apple documents a 0-35 C ambient operating range for iPhone/iPad and warns that high temperatures can permanently reduce battery capacity; charging may slow or stop when the device is too warm. [S11] Other manufacturers publish their own limits, which should be checked per model.

Sources of heat

Charging losses, display use, CPU/GPU work, radios and tightly packed devices all add heat. A rack near a window or inside a closed cabinet can run materially hotter than the room. Measure at phone inlet locations, not only at the HVAC thermostat.

Continuous 100% charge is also undesirable for battery aging. Where the manufacturer provides optimized charging or managed battery features, evaluate them. A fleet can be scheduled to maintain adequate charge without forcing every battery to remain at maximum state of charge continuously.

- Ambient intake temperature.
- Rack exhaust temperature.
- Representative battery/device temperature where accessible.
- Fan operation and dust loading.
- Thermal throttling or charging-paused events.

Airflow

Use a predictable front-to-back or bottom-to-top flow. Recirculation is the enemy: if hot exhaust immediately returns to the intake side, adding more fans can increase noise without improving temperature. Baffles or spacing can be more effective than raw fan count.

Design alerts well below absolute device limits so operators have time to react. The correct warning threshold depends on the device model and sensor being monitored.

Engineering checklist

- Baseline temperatures at 25%, 50% and 100% rack occupancy.
- Repeat test with all devices charging from low battery.
- Define warning and critical thresholds.
- Test a failed-fan scenario.
- Keep rack away from direct solar load.

11. Fire/electrical safety and UPS

A phone rack combines many lithium-ion batteries, charging electronics and AC connections. Use commercially rated electrical components, inspect cords and connectors, and follow local electrical/fire rules. OSHA guidance for extension cords emphasizes proper construction, grounding where required, strain relief and inspection; local regulations may impose additional requirements. [S31]

Safety principles

Do not treat consumer power strips as permanent high-density distribution without verifying their ratings and local code suitability. Do not defeat grounding or overcurrent protection. Remove swollen, damaged or unusually hot batteries from service immediately according to manufacturer and local hazardous-waste guidance.

Provide a clearly reachable main disconnect for the rack and an incident procedure for smoke, odor, battery swelling or abnormal heat. Staff should know not to puncture or compress a damaged lithium-ion battery.

- Rated PDU / distribution.
- Cable inspection schedule.
- No blocked ventilation around chargers.
- Smoke detection appropriate to the site.
- Fire response plan aligned to local authority guidance.

UPS strategy

Prioritize graceful operation rather than attempting to run all charging for hours. A practical UPS can keep router, firewall, switch, APs, controller and database online long enough to preserve state and shut down cleanly. Phones already contain batteries and may not need to be on UPS-backed charging circuits unless continuity requirements demand it.

Calculate UPS VAW load from measured equipment and required runtime. Validate actual runtime with a controlled test after installation.

12. Host computers and local controllers

The local controller is the bridge between the physical fleet and the backend. It may host ADB, local media caching, telemetry collectors, a message broker edge client, a web dashboard, or Apple provisioning utilities.

Sizing

CPU requirements are modest for orchestration but can rise sharply if the host transcodes video, performs local AI analysis or handles many concurrent media transfers. RAM should cover the OS, device tooling, database/cache agents and headroom. Use NVMe storage if the host is a media staging point; otherwise keep large media in object storage or a dedicated NAS.

For a 10-25 device pilot, one reliable workstation or mini-PC may be enough. At higher scale, separate the fleet controller from heavy video processing and from the primary database to reduce blast radius.

- At least two network interfaces if you want clean management/device separation.
- Automatic startup of required services after reboot.
- Disk encryption.
- Admin accounts protected by MFA.
- Regular image/configuration backup.

Operating system choice

Android ADB works across Windows, macOS and Linux. Apple Configurator is a macOS tool, while long-term Apple fleet control should be performed with a suitable MDM rather than requiring every device to remain tethered to one Mac.

Choose the host OS your operators can patch and support reliably. Consistency beats novelty.

13. Inventory, labeling, spares and lifecycle

A fleet becomes manageable when physical identity and database identity are never ambiguous. Build inventory before scale.

Minimum asset record

Each device should have an internal asset ID, manufacturer/model, serial/IMEI where appropriate, purchase source/date, warranty status, OS version, storage, battery-health note, rack/slot, assigned creator or pool, management-enrollment status and retirement date. Sensitive identifiers should be access-controlled.

Do not encode creator names or other sensitive information directly into externally visible device labels. Use opaque asset IDs and resolve them in the management system.

- Asset barcode/QR.
- Cable and charger-port mapping.
- Spare status: ready / repair / quarantine / retired.
- Chain-of-custody when handed to an employee.

Lifecycle

Use a standard state machine: Received -> Inspected -> Enrolled -> Available -> Assigned -> Maintenance -> Quarantined -> Retired -> Wiped/Disposed. Every transition should be timestamped and attributable.

Maintain spare capacity. A fleet that operates at 100% device utilization has no room for OS updates, battery replacement or incident quarantine.

PART III - NETWORKING, CONNECTIVITY, PROXIES AND TUNNELS

Networking is where “phone farm” discussions often become confused. Public IP address, local Wi-Fi address, DNS resolver, proxy, VPN tunnel, SIM carrier and device identity are different concepts. This part separates them and shows how to design reliable, auditable egress without using the network to misrepresent identity or defeat enforcement.

14. Network reference architecture

The recommended baseline is a business firewall/router, managed switching, one or more centrally managed access points, separate VLANs/SSIDs for fleet devices and management, and explicit DNS/egress policy.

Network Topology

Segmentation keeps device traffic, management and storage independently controllable

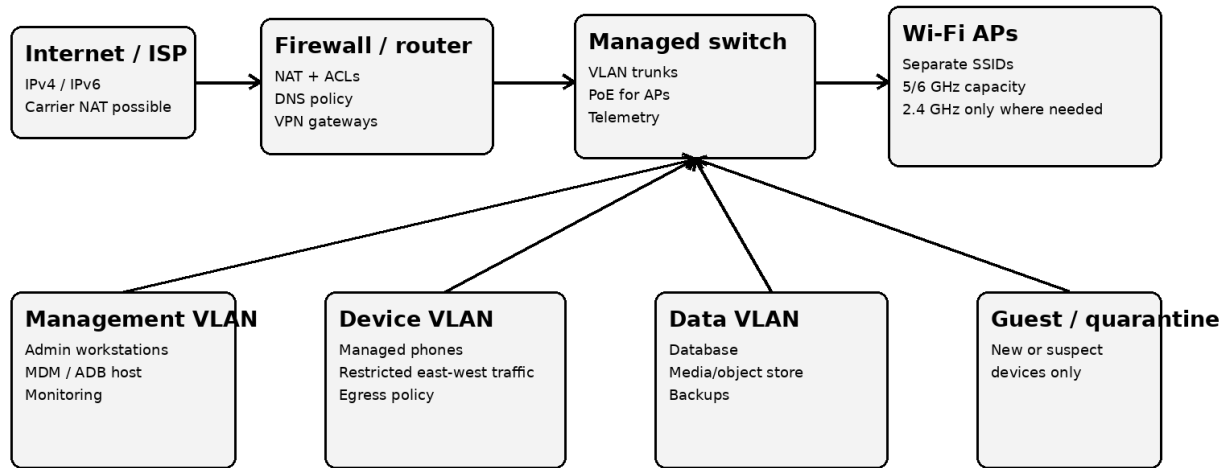


Figure 14-1. Segmented network reference.

Zones

The management zone contains administrator workstations, MDM connectors, ADB hosts and monitoring. The device zone contains phones. The data zone contains database, media store and backups. A quarantine/guest zone can isolate new or suspect devices. ACLs should permit only the flows required between zones.

Segmentation reduces the impact of a compromised phone or stolen employee credential. It also makes troubleshooting easier because you know which traffic class a packet belongs to.

- Management VLAN.
- Device VLAN(s).
- Data/storage VLAN.
- Guest/quarantine VLAN.
- Optional VoIP/office user VLAN separate from the farm.

Internet edge

The firewall should log high-level connection and security events without collecting more personal data than necessary. Where a business VPN gateway or proxy is used, document which devices/apps use it and why.

Prefer deterministic configuration to ad-hoc per-phone settings. Configuration drift is one of the main reasons ten devices behave differently despite looking identical.

15. Wi-Fi radio and capacity planning

Phone density is a radio-capacity problem, not merely a “strong signal” problem. High-density wireless design requires enough airtime, channel planning and AP placement for the concurrent workload. Vendor high-density design guides emphasize planning client density and bandwidth rather than relying on maximum advertised AP client counts.

2.4 vs 5/6 GHz

2.4 GHz travels farther but has less clean spectrum and more interference. 5 GHz provides more channels and is usually preferable for dense phone racks when devices and region support it. 6 GHz can add capacity on compatible devices and APs but has different propagation and regulatory constraints.

Do not put a rack of many phones directly against one AP and assume proximity guarantees capacity. Very strong signals can coexist with contention. Use multiple APs only when channel planning supports it; simply adding APs on overlapping channels can make performance worse.

- Measure channel utilization and retry rates.
- Use wired Ethernet backhaul for APs.
- Disable legacy rates/features only after confirming device compatibility.
- Set reasonable transmit power rather than maximum everywhere.
- Perform throughput and latency tests from representative rack positions.

SSID design

Keep SSID count low because management overhead consumes airtime. Separate networks by security/policy need, not by creating one SSID per creator. VLAN assignment can provide segmentation behind a smaller number of SSIDs.

If using ADB wireless debugging, the workstation and device must meet the applicable pairing/network requirements. Android 11+ supports wireless debugging with pairing; use it on trusted networks only. [S3]

Metric	What it indicates	Typical response
Channel utilization	Airtime pressure	Reduce load / add properly planned capacity
Retry rate	Interference / weak link	RF survey, power/channel tuning
RSSI/SNR	Link quality	AP placement / obstacles
Latency/jitter	Congestion or WAN issue	Separate LAN vs WAN tests
Roaming events	AP handoff behavior	Tune only if devices actually move

16. Ethernet, switches and VLANs

All infrastructure that can be wired should generally be wired. APs, controller hosts, storage and servers should use Ethernet so wireless airtime is reserved for phones.

Switching

A managed switch allows VLAN trunks, per-port statistics, PoE power for APs, and port isolation where useful. Maintain a port map so a failed AP or controller can be found quickly.

For larger installations, redundant uplinks and a second switch may be justified, but complexity should follow actual uptime requirements.

- Label patch cables at both ends.
- Use separate management access for switches/APs.
- Back up switch configuration.
- Disable unused physical ports or place them in a restricted VLAN.

VLAN/ACL design

A VLAN is a Layer-2 segmentation mechanism; it is not security by itself. Inter-VLAN routing rules on the firewall or Layer-3 switch enforce which systems can communicate. A device VLAN typically needs DNS, NTP, internet access and access to explicitly approved internal services, not unrestricted access to the management network.

Block unsolicited device-to-device traffic unless the workflow requires it. This limits lateral movement if one handset is compromised.

17. IPv4, IPv6, NAT and CGNAT

Understanding address translation prevents mistaken assumptions about “one IP per phone.” A phone normally receives a private local IP address from DHCP. The router may translate many private IPv4 addresses into one or several public IPv4 addresses using NAT. A mobile carrier may perform another layer of carrier-grade NAT (CGNAT), meaning many subscribers share public IPv4 space.

IPv4/NAT

Private IPv4 addresses such as RFC1918 ranges are not globally routable. The edge router keeps connection state so return traffic is mapped back to the correct phone. NAT changes addressing; it does not inherently anonymize application identity.

Port/address translation also means a platform may see the same public egress IP for many legitimate devices behind a business connection. Conversely, the same phone may appear behind different public addresses as networks change.

CGNAT and mobile networks

CGNAT performs NAT inside the carrier network at large scale. This is common where public IPv4 addresses are scarce. IPv6 can provide globally routable addresses without the same NAT requirement, though firewalls still control inbound/outbound connectivity. [S18]

Do not design identity assumptions around IPv4 alone. Modern applications can use IPv4 and IPv6 depending on network and DNS responses.

- Record whether each segment has IPv6 enabled.
- Test application behavior over IPv4 and IPv6.
- When troubleshooting “wrong IP,” identify the local IP, router WAN IP and externally observed IP separately.

18. DNS and resolver design

DNS converts hostnames into addresses and is part of both reliability and privacy. Traditional DNS can be visible to the local network; DNS over HTTPS (DoH) and DNS over TLS (DoT) encrypt queries between client and resolver. [S17]

Resolver choices

A business can use the firewall as a forwarding resolver, an internal resolver, or a managed external resolver. The goal is consistent policy, fast failure diagnosis and appropriate logging. Do not mix several ad-hoc per-phone resolver settings unless there is a documented requirement.

If a VPN is used, decide whether DNS should also traverse the tunnel. Split DNS may be necessary for internal names. DNS leaks are not merely a “privacy hack” topic; mismatched routing can break corporate resources and make incident analysis confusing.

- Monitor resolver latency and failure rate.
- Use DNSSEC-validating resolvers where suitable.
- Keep internal hostnames out of public DNS.
- Minimize query logging retention to operational/security needs.

Failure diagnosis

Test in layers: can the phone reach the default gateway, can it reach a known IP, can it resolve a hostname, can it establish TLS to the target, and can the application authenticate? This avoids blaming “the proxy” for a DNS or certificate problem.

19. Forward proxies and proxy protocols

A forward proxy is an intermediary that makes outbound requests on behalf of clients. For HTTP(S), a proxy may forward requests directly or use the CONNECT method to create a TCP tunnel for TLS traffic. [S14] A proxy is different from a VPN: a proxy is usually application/protocol-aware, while a VPN normally routes IP packets through a tunnel.

Proxy, VPN and Egress Concepts

Networking tools are useful for security and segmentation; they do not legitimize identity deception

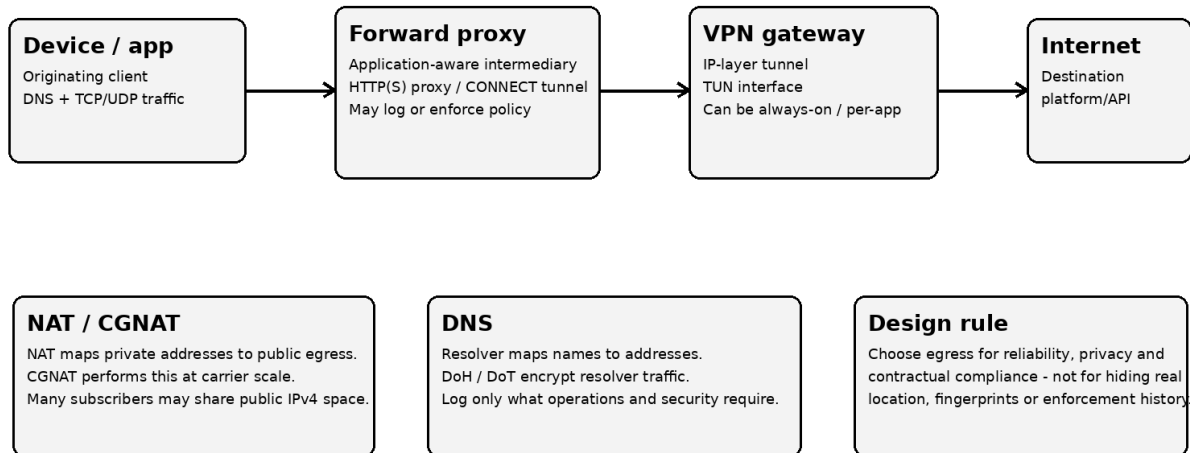


Figure 19-1. Proxy, VPN, NAT and DNS are separate layers.

Why businesses use forward proxies

Legitimate uses include outbound policy enforcement, malware filtering, controlled egress, audit logging, caching in some environments, and allowing specific internal applications to reach the internet through a known gateway.

On mobile devices, proxy support varies by application and OS. A Wi-Fi proxy configured at the network level may not affect every protocol. A VPN-based egress design is often more comprehensive when the requirement is “all traffic from these apps/devices follows this path.”

- HTTP proxy: understands HTTP semantics.
- CONNECT: creates a tunnel commonly used for HTTPS through an HTTP proxy.
- SOCKS-style proxy: relays TCP and sometimes UDP without understanding HTTP content.
- Transparent/intercepting proxy: network redirects traffic; introduces certificate/privacy complexity for TLS inspection.

Proxy provenance and risk

Marketing terms such as “datacenter,” “residential” and “mobile” refer to how the public egress address is sourced. They do not create a compliance exemption. Third-party proxy providers introduce security, privacy, availability and contractual risks because they may observe metadata, alter routes or lose access to address pools.

This manual does not recommend proxy rotation patterns or provider selection for avoiding platform detection. If an agency needs a proxy, choose it for an articulated business purpose such as security, regional testing with authorization, or controlled corporate egress, and perform legal/provider due diligence.

Engineering checklist

- Document why a proxy is needed.
- Confirm which apps/protocols actually use it.
- Test TLS/certificate behavior.
- Measure latency and failure rate.
- Define what the provider can log and retain.
- Create a rollback path to direct egress.

20. VPNs, tunnels and per-app routing

A VPN creates a logical network tunnel between device and gateway. On Android, VpnService creates a virtual TUN interface, allowing a VPN application to read outgoing IP packets and return incoming packets through the interface. Android supports always-on VPN and per-app allow/disallow lists. [S4]

Use cases

VPNs are appropriate for securing traffic over untrusted networks, reaching private corporate services, isolating a fleet behind managed gateways, and applying consistent egress/security policy. WireGuard is one modern protocol designed as a secure network tunnel; enterprise deployments may also use IPsec/IKEv2 or vendor clients. [S19]

Per-app routing can keep only managed applications inside a corporate tunnel while leaving OS update or other traffic direct, or it can do the reverse depending on policy. Test carefully because platform apps may depend on push services, CDNs and media endpoints outside a simple hostname list.

- Always-on mode for high-assurance corporate access.
- Kill/block mode if non-VPN traffic must not bypass policy.
- Per-app lists to scope tunnel use.
- Gateway redundancy for production.
- Key rotation and device revocation.

What a VPN does not do

A VPN changes the network path and may change the public IP address observed by a service. It does not change the account's history, app telemetry, device security state, content, user behavior or the platform's rules. Do not treat it as a method for falsifying identity or location.

21. Cellular, SIM and eSIM concepts

Cellular connectivity can provide redundancy or legitimate mobile testing, but it adds carrier contracts, SIM inventory, signal variation and CGNAT behavior. A mobile plan should be procured and used in accordance with carrier terms and local identity/registration laws.

Physical SIM vs eSIM

Physical SIMs are easy to move between compatible devices but create a physical inventory problem. eSIM can simplify provisioning on supported carriers/devices but may require carrier portals or activation workflows. In either case, map the subscription to the device and cost center in inventory.

Do not confuse the subscriber identity on a SIM with the identity of a social account. Keep telecom records and platform credentials in separate systems.

- Carrier name/plan.
- SIM/eSIM identifier stored with restricted access.
- Assigned device.
- Monthly data cap and roaming rules.
- APN/VPN requirements.
- Suspension/termination procedure.

Redundancy

For business continuity, a cellular router can provide WAN failover for the site. This is often easier to manage than giving every phone its own cellular plan. Individual phone cellular service should exist only where the workflow needs it.

22. Egress observability and troubleshooting

Operators need to answer: which path did this device use, which DNS resolver answered, what public IP was observed, did the VPN/proxy work, how much latency was introduced, and where did the request fail? Build observability for those questions without recording sensitive content unnecessarily.

Telemetry

Collect device ID, network profile, SSID, gateway reachability, resolver status, tunnel status, public egress test result, latency, packet loss and timestamp. Avoid storing credentials, message contents or unnecessary browsing history in network logs.

Use synthetic probes to your own endpoints for health checks. Do not continuously probe third-party platforms in a way that creates abusive load.

- DHCP lease and VLAN.
- VPN/proxy state.
- DNS response time.
- WAN packet loss.
- Gateway CPU/connection utilization.
- AP client count/channel utilization.

Troubleshooting ladder

Always isolate layers: device hardware -> Wi-Fi association -> DHCP -> gateway -> DNS -> tunnel/proxy -> TLS -> application authentication -> platform/API response. A documented ladder reduces random configuration changes that create new faults.

23. Network failure modes

Most fleet networking incidents are ordinary infrastructure failures: overloaded Wi-Fi, dead cables, DNS outages, expired tunnel credentials, saturated WAN links or misapplied ACLs.

Common scenarios

If all devices on one rack fail simultaneously, look for shared AP, switch, VLAN, DHCP or power dependencies. If only one application fails but other internet access works, look above the IP layer. If a subset of devices intermittently disconnects, inspect RF utilization and client-specific power-saving behavior.

When changing proxy/VPN configuration, deploy to a small ring first. A bad egress policy can strand the entire fleet from MDM or management services.

Resilience patterns

Dual WAN, redundant DNS resolvers, spare APs, configuration backups and staged changes provide more benefit than exotic networking. Keep a direct, secure management path so a failed content egress tunnel does not remove your ability to manage the device.

- Configuration versioning.
- Change tickets with rollback.
- Canary device/rack.
- Out-of-band admin path where justified.
- Post-incident review.

Failure	Blast radius clue	First containment
AP failure	One RF cell/rack	Move clients / replace AP
DHCP failure	New joins fail	Restore DHCP / lease service
DNS failure	IP works, names fail	Fail over resolver
VPN gateway failure	Tunnel users only	Fail over or controlled bypass
WAN outage	Whole site	Secondary ISP/cellular failover

PART IV - DEVICE FLEET MANAGEMENT

Fleet management should make devices boring: repeatable enrollment, predictable settings, known application versions, controlled access and fast recovery. Apple and Android require different approaches.

24. Apple fleet architecture

Apple's enterprise deployment model centers on organization ownership, Automated Device Enrollment where available, supervision and Mobile Device Management (MDM). Apple Platform Deployment documents the lifecycle from planning and enrollment through configuration and ongoing management. [S5]

Apple Fleet Control Flow

Supervision + MDM for organization-owned iPhone/iPad fleets

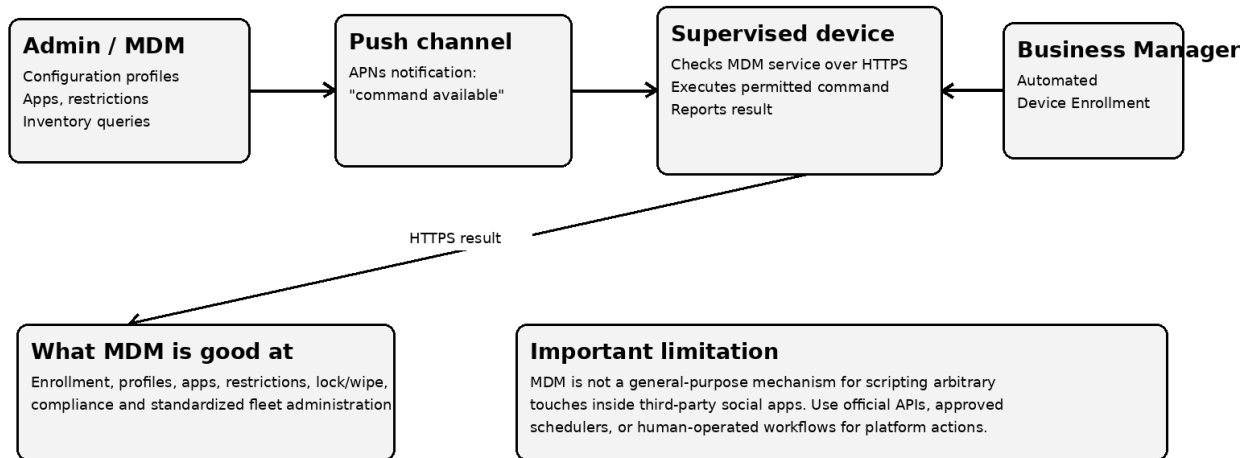


Figure 24-1. Simplified Apple MDM control flow.

Core components

Apple Business Manager (or the appropriate Apple organizational service) establishes organization ownership and can assign devices to an MDM service. Supervision provides additional management controls for organization-owned devices. The MDM service sends configuration profiles, app assignments and commands while devices communicate with Apple push infrastructure and the MDM service.

Use a commercial or internally operated MDM that explicitly supports the commands, restrictions and app deployment you require. The phone rack itself should not depend on one operator manually configuring every iPhone.

- Organization account and administrative roles.
- Device enrollment/assignment.
- Supervision.
- MDM server/service.
- APNs connectivity.
- Managed app distribution.

Design principle

MDM is the administrative plane. It is not a generic remote-control framework for simulating user gestures in third-party applications. Keep publishing workflows separate.

25. Supervision, enrollment and MDM

Supervision signals that an iPhone/iPad is organization-controlled and unlocks additional management capabilities. Automated Device Enrollment can supervise eligible organization-owned devices during setup; Apple Configurator can be used in certain manual preparation workflows. [S5]

Enrollment flow

At acquisition, record the serial and organization ownership. Assign the device to the intended MDM, boot/reset it, complete Automated Device Enrollment, verify supervision, apply baseline profiles, install managed applications, test connectivity, then mark the device Ready in inventory.

MDM commands typically use push notification to tell the device that a command is available; the device then contacts the management service over HTTPS, executes supported actions and reports results. [S6][S7]

- Wi-Fi/certificate profiles.
- Passcode and security policy.
- Managed apps.
- Restrictions appropriate to role.
- Lost mode/lock/wipe capability where supported.
- Inventory and compliance reporting.

Recovery

If a supervised device is lost or compromised, use the MDM's supported lock/wipe workflow and revoke relevant credentials. If it is reassigned, wipe and re-enroll rather than merely signing out of one social application.

26. Android Enterprise architecture

Android Enterprise provides management modes for personally owned and company-owned devices. For a phone farm, company-owned fully managed devices are normally the cleanest model because the organization can manage the complete work-only device. Dedicated devices are a subset that can be locked to a small application set. [S1]

Android Fleet Control Flow

Android Enterprise for policy; ADB for authorized engineering and diagnostics

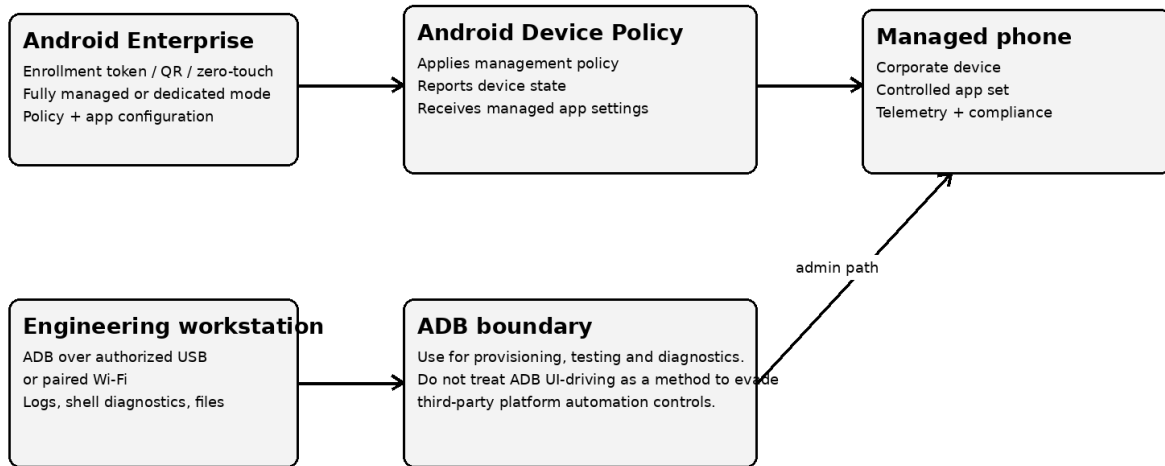


Figure 26-1. Android Enterprise plus authorized ADB engineering.

Android Management API

Google's Android Management API uses Android Device Policy on the device to receive and enforce enterprise policies. Provisioning creates a device object bound to the enterprise. Enrollment tokens determine ownership/management mode; company-owned devices can use fully managed configurations. [S1]

Policies can control application installation, permissions, restrictions, networking settings, system updates and compliance behavior depending on Android version and management mode.

- Enterprise resource.
- Enrollment tokens.
- Policies.
- Device objects and state.
- Managed Google Play application policy.

Dedicated vs fully managed

Dedicated mode is suitable when the phone should expose only a controlled application set. If operators need several social apps, browser, camera, authenticator and internal tools, a normal fully managed configuration may be more practical than an overly restrictive kiosk.

27. Android provisioning and policy

Google documents QR, zero-touch, NFC and other provisioning methods. For company-owned work-only devices, full management allows the enterprise to manage all applications and enforce broad policy. [S1]

Pilot provisioning

For a small pilot, QR enrollment is straightforward: factory-reset the corporate device, enter the Android provisioning flow, scan the organization-generated enrollment QR, allow Android Device Policy to complete setup, then verify policy/application state. At scale, eligible zero-touch procurement reduces hands-on setup.

Enrollment tokens should be short-lived where practical and protected as administrative secrets because possession can allow a device to enroll into the enterprise configuration.

- Baseline Wi-Fi or setup network.
- Enrollment token handling.
- Policy assignment.
- Required apps.
- VPN client if required.
- Compliance validation.

Policy rings

Use Stable, Canary and Quarantine policies. Canary receives new OS/app/policy changes first. Stable is production. Quarantine removes unnecessary access while preserving enough management connectivity for investigation.

28. ADB for authorized engineering

Android Debug Bridge (ADB) is Google's command-line bridge between a development workstation and Android devices. It can communicate over USB and, on modern Android, paired wireless debugging. [S3] It is appropriate for provisioning support, logs, diagnostics and controlled testing on devices you own or are authorized to administer.

Architecture

ADB consists of a client, local server and device daemon. A workstation can enumerate authorized devices, open a shell, collect logs and transfer files. USB is usually the most predictable for a rack; wireless debugging reduces cables but depends on trusted Wi-Fi and pairing state.

Use a dedicated engineering account and workstation. Disable debugging on production devices if it is not needed, or restrict the workstation/network that can reach it.

- Inventory verification.
- Log collection.
- File push/pull for internal test assets.
- Package/version diagnostics.
- Network diagnostics.

Boundary

ADB can technically generate input events and interact with applications. This manual does not provide a workflow for using ADB to automate third-party social UIs where automation is prohibited or to make automated behavior appear human. Use supported APIs or human operation for those actions.

29. App distribution and configuration

A managed fleet needs predictable application versions. Install apps through MDM/managed app channels where available, define which apps are required, and avoid ad-hoc APK or IPA distribution unless you own the software and have an authorized enterprise development path.

Application catalog

Maintain a catalog with app name, package/bundle ID, platform, source, minimum version, required permissions, owner, data classification and update ring. Social apps, password manager, authenticator, VPN client, internal task app and device-health agent should each have an owner.

Permissions should be the minimum needed. For example, a publishing app may need photo-library access but not contacts. Review permissions after major app updates.

- Required vs optional.
- Version and update policy.
- Permissions.
- Data stored locally.
- SSO support.
- Uninstall/retirement behavior.

Internal companion app

Consider a small internal “operator companion” app that shows device ID, assigned creator/account, task queue, approved asset links, check-in status and incident button. It should not store social-account passwords and should not covertly drive other applications.

30. OS updates and maintenance rings

Ignoring OS updates creates security debt; updating every device immediately creates operational risk. Use rings.

Rings

Canary devices receive updates first and run representative workloads. After a defined observation window, a larger pilot ring updates, then production. Critical security fixes may justify accelerated rollout after compatibility testing.

MDM or Android Enterprise update controls can help schedule or defer updates within platform-supported limits. Do not leave devices indefinitely frozen on unsupported releases.

- Canary: 1-5% of fleet.
- Pilot: 10-20%.
- Production: remaining devices.
- Emergency: accelerated security rollout with explicit approval.

Maintenance window

Drain publishing tasks from the device, confirm battery/power, install update, reboot, verify management heartbeat, verify required apps and network, then return to Ready. Record the new build version automatically where possible.

31. Health telemetry and remote actions

A fleet dashboard should answer “what is healthy right now?” without requiring someone to walk down the rack.

Health model

At minimum track last heartbeat, OS/app version, storage free, battery/charging state, management compliance, network profile, last task, last error and current assignment. Temperature data availability varies by OS/device; use rack sensors even when device-level telemetry is limited.

Status should be derived, not subjective: Ready, Busy, Maintenance, Degraded, Quarantined, Offline.

- Heartbeat age.
- Charging state.
- Free storage.
- Wi-Fi/VPN state.
- Required app versions.
- MDM/Enterprise compliance.
- Task error rate.

Remote actions

Administrative actions can include lock, wipe, restart where supported, reapply policy, request inventory, rotate internal app tokens, or move a device to quarantine. High-impact actions should require stronger authorization and be written to the audit log.

PART V - SOFTWARE CONTROL PLANE

The coordinator makes the phone farm a system rather than a pile of devices. It should model devices, creators, accounts, assets, approvals, tasks, results, networks and employees as separate entities with an auditable workflow.

32. Control-plane architecture

A practical control plane consists of an API service, relational database, object/media storage, task queue or broker, device/fleet connectors, authentication/RBAC, metrics/logging and an operator web interface.

Reference Architecture

A compliant, observable creator-traffic operations platform

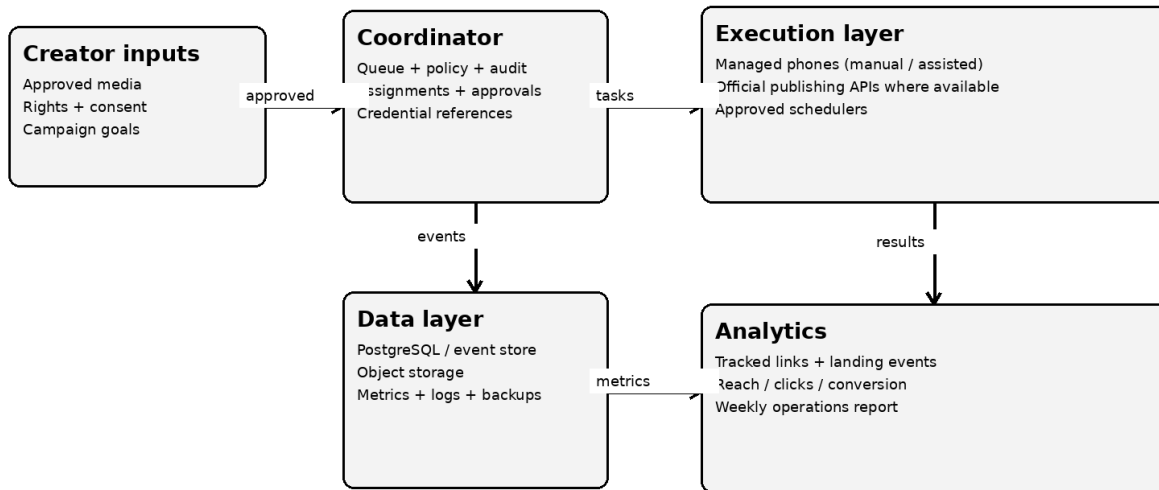


Figure 32-1. Control plane and execution layers.

Service boundaries

The API service owns business rules. The relational database stores durable entities and task state. Object storage holds media and large artifacts. A broker or queue delivers asynchronous work. Device connectors translate generic management intents into Apple MDM, Android Enterprise, internal companion-app or other supported actions.

Keep platform publishing connectors separate from device-management connectors. This prevents a social API outage from breaking fleet administration and makes authorization boundaries clearer.

- API/auth service.
- PostgreSQL or equivalent relational store.
- Object storage.
- Queue/broker.
- Fleet connectors.
- Publishing connectors.
- Analytics/event pipeline.
- Web dashboard.

Deployment

For a pilot, these can run on one secured server using containers. For production, separate stateful services, backups and secrets. Avoid unnecessary microservices: operational complexity grows faster than device count.

33. Device-to-coordinator communications

There are two broad channels: enterprise management channels provided by Apple/Google, and a custom internal agent/companion channel controlled by the agency. Do not reinvent MDM for functions MDM already performs.

HTTPS model

An internal companion app can periodically call an HTTPS endpoint to register heartbeat, fetch permitted task metadata and report completion. Use short-lived OAuth-style tokens or device certificates rather than embedding a shared static API key in every phone.

For near-real-time updates, WebSocket or server-sent events can be added, but simple polling is often easier to operate at 10-100 devices. Poll intervals should use jitter so all devices do not hit the server simultaneously.

- TLS for all remote traffic.
- Unique device identity.
- Token rotation/revocation.
- Idempotent task receipt.
- Clock synchronization.
- Backoff on failure.

Message broker

MQTT is a lightweight publish/subscribe protocol widely used for constrained devices and IoT-style messaging. [S20] A companion app could publish health telemetry and subscribe to a narrow command topic, but do not expose a broker directly to the public internet without strong authentication, authorization and TLS.

A queue/broker is transport, not the source of truth. Durable task state belongs in the database.

34. Task queue and state machine

Every action should move through explicit states so operators know whether it is waiting, running, verified or failed. Silent retries hide problems and can create duplicate posts.

Task Lifecycle

Every publishing or device action should have an accountable state transition

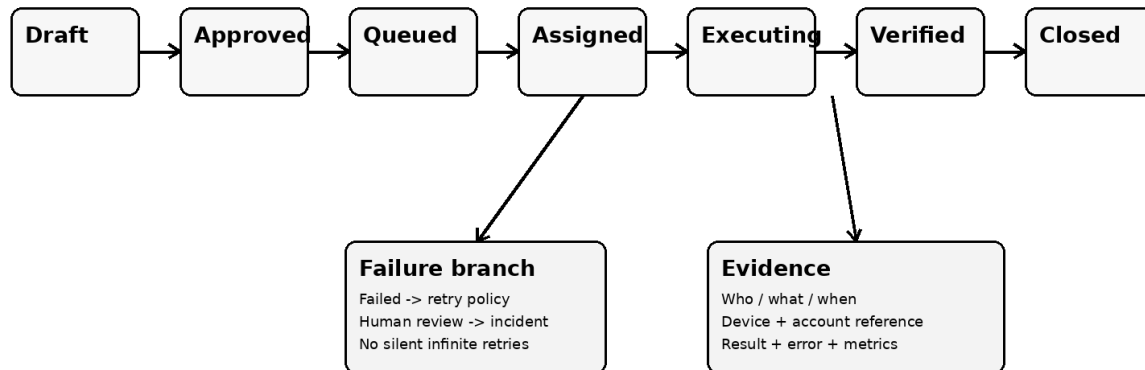


Figure 34-1. Auditable task lifecycle.

Reference states

Draft: not yet approved. Approved: content/action permitted. Queued: eligible for scheduling. Assigned: bound to executor/device/operator. Executing: in progress. Verified: expected result observed. Closed: complete and immutable except for annotations. Failed tasks enter a review/retry branch with a reason.

Use idempotency keys for API publishing so a network retry does not create a second post. Human tasks should require the operator to record the platform post ID or a verification artifact before closure.

- Maximum retry count.
- Retryable vs terminal error classes.
- Deadline/SLA.
- Priority.
- Executor type.
- Required approval IDs.
- Verification method.

Scheduling

Scheduler logic should consider platform/account restrictions, operator shift, device readiness, campaign windows and content approvals. It should not create prohibited spam by blindly maximizing throughput.

35. Database and entity model

The database should model real business entities instead of embedding everything in a spreadsheet row. This supports permissions, history and reporting.

Core Data Model

Keep identities, assets, devices, accounts and tasks as separate entities

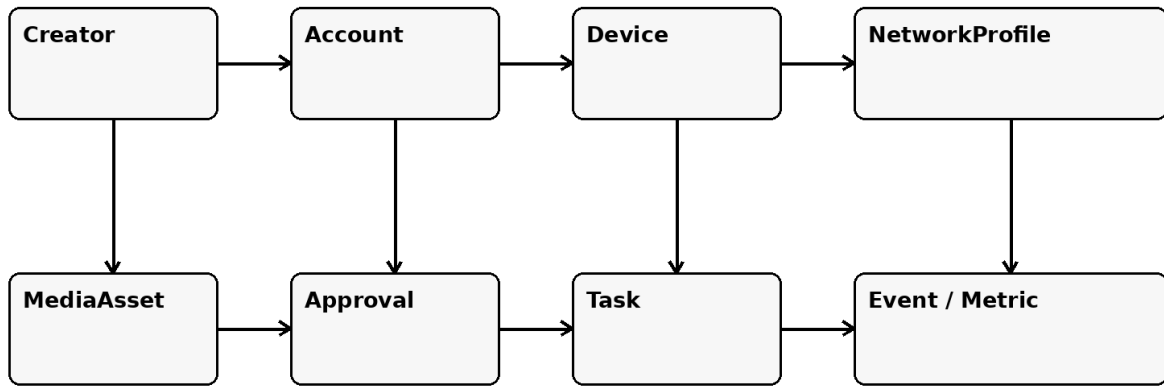


Figure 35-1. Core entity separation.

Core entities

Creator represents the contracted person/business. Account represents one platform account and stores ownership/authorization metadata, not plaintext credentials. Device represents a physical asset. NetworkProfile describes approved egress configuration. MediaAsset stores immutable asset metadata and storage URI. Approval links a person to a permitted action/asset. Task describes work. Event/Metric stores observed outcomes.

Use stable UUIDs internally. Human-readable labels can change; primary identity should not.

- Creator.
- PlatformAccount.
- Device.
- NetworkProfile.
- MediaAsset / AssetVersion.
- Approval.
- Task / TaskAttempt.
- Operator.
- AuditEvent.
- MetricEvent.

History

Use append-only audit records for security-relevant changes. For mutable business records, store updated_at and updated_by; for especially sensitive fields keep version history or event-sourced change records.

36. Object/media storage

Creator media is valuable and sensitive. Centralize it in access-controlled object storage rather than uncontrolled phone camera rolls and employee downloads.

Storage model

Use immutable source assets plus derived versions. Store SHA-256 or another strong checksum for integrity, creator ID, usage authorization, source, creation/import timestamp, classification and retention/expiration. A derived version should point back to its source and transformation recipe.

Phones should download only the assets required for assigned tasks, ideally through time-limited signed links. Clear local staging areas on reassignment/offboarding where technically possible.

- Server-side encryption.
- Restricted buckets/containers.
- Versioning where useful.
- Lifecycle/retention rules.
- Backup/replication.
- Access logging.

Content safety

Only store adult creator content for verified adults with documented authorization and applicable consent/releases. Build a takedown/revocation process that can locate every derived copy from an asset ID.

37. API design

The internal API should expose business intentions, not raw device-control primitives. “Assign approved asset X to account Y” is safer than “tap coordinate 400,900.”

Resource examples

Use REST or another consistent RPC model with resources such as `/devices`, `/accounts`, `/assets`, `/approvals`, `/tasks`, `/events` and `/metrics`. Every write should authenticate the caller and enforce role/creator scope.

Use request IDs and idempotency keys. Return machine-readable error codes. Record actor, source IP/device, target object and result in audit logs for security-relevant writes.

- GET `/devices/{id}`.
- POST `/tasks`.
- POST `/tasks/{id}/approve`.
- POST `/tasks/{id}/assign`.
- POST `/tasks/{id}/result`.
- GET `/accounts/{id}/metrics`.

Versioning

Version the API contract before external clients depend on it. Internal rapid changes are easier if the companion app and backend can tolerate one previous schema version during rolling updates.

38. Messaging and event transport

Asynchronous messaging decouples slow device/network actions from interactive web requests. A user should not keep an HTTP request open while waiting for a phone task to finish.

Queue semantics

At-least-once delivery is common and requires idempotent consumers. Exactly-once behavior is usually implemented at the application level using unique keys and database constraints rather than assumed from the broker.

Use separate queues/topics for device commands, publishing jobs, analytics ingestion and low-priority maintenance so one workload cannot starve another.

- Dead-letter queue.
- Retry delay/backoff.
- Maximum age.
- Consumer concurrency.
- Per-account/device serialization where necessary.

MQTT topic example

For telemetry, a namespace such as `fleet/{device_id}/health` and `fleet/{device_id}/events` is easy to authorize. Commands should be narrow and typed. Do not create a universal remote-shell topic in a production broker.

39. Metrics, logs and traces

Prometheus is a common metrics and alerting system built around time-series data and label dimensions. [S21] Whether using Prometheus or another stack, distinguish metrics, logs and traces.

Metrics

Metrics answer aggregate questions: online device count, task success ratio, queue depth, API latency, Wi-Fi loss, VPN tunnel availability. Keep label cardinality controlled; do not use a unique post URL as a Prometheus label if it creates millions of series.

Logs answer event details: device enrolled, task failed, credential reference rotated. Traces connect calls across services when debugging distributed workflows.

- Fleet online %.
- Task success/failure by executor.
- Queue age P50/P95.
- Publishing connector error rate.
- Database latency.
- Storage capacity.
- Network health.

Alerting

Alerts should be actionable: ">20% of devices in rack B missed three heartbeats" is useful; "CPU 71%" may not be. Add runbook links to alerts.

40. Credential and secret handling

Credentials are one of the highest-value assets in an OFM system. Do not store platform passwords, session cookies, MFA seeds or API keys in source code, shared spreadsheets, task descriptions or chat channels.

Vault model

Use a password/secret manager that supports per-user access, audit logs, MFA and revocation. The application database stores a reference such as `vault://creator/123/platform/x` rather than the secret value. Backend services receive only the secret they need, ideally for a limited time.

Where a platform supports delegated access, teams/business accounts or OAuth, prefer that over sharing the creator's primary password.

- Unique operator accounts.
- MFA/hardware keys for administrators.
- No shared root credential.
- Recovery codes stored securely.
- Immediate revocation on offboarding.
- Periodic access review.

Session management

Social apps may maintain long-lived sessions on phones. Treat the device itself as a credential-bearing asset: lock it, encrypt it, inventory it, and wipe/revoke on loss or reassignment.

41. Availability, backup and disaster recovery

The control plane should survive ordinary failures without losing authorization records, task history or media.

Backup scope

Back up relational databases, configuration, audit records, object-storage metadata and essential media according to business/legal requirements. Secrets should be backed up through the vault's supported recovery mechanism, not exported into general backups.

Test restore. A backup that has never been restored is an assumption, not a recovery plan.

- Define RPO: acceptable data loss window.
- Define RTO: acceptable service recovery time.
- Keep at least one backup copy isolated from primary credentials.
- Document dependency order: identity -> database -> object store -> queue -> API -> workers.

Degraded operation

If the coordinator is down, operators should have a clear rule: pause new publishing or use an approved manual fallback with later reconciliation. Do not create a shadow spreadsheet that permanently bypasses audit and approval controls.

PART VI - OFM OPERATIONS, CONTENT AND ANALYTICS

The technical fleet exists to support a controlled creator-content workflow. This part focuses on authorization, media, publishing, employees and measurement.

42. Creator onboarding and authorization

Before an account or device is assigned, the creator relationship should be documented and operationally represented in the system.

Onboarding packet

Record legal/business identity as required by the agency, contract reference, platform account ownership, recovery-channel ownership, allowed content categories, geographic/commercial restrictions, approved employees, revenue attribution identifiers and exit procedure. Sensitive identity documents require stricter access than routine content metadata.

The creator should know which agency systems and people will access which accounts. Authorization should be revocable.

- Contract/authorization.
- Account inventory.
- Recovery contacts.
- Content rights/consent.
- Brand rules.
- Escalation contacts.
- Offboarding instruction.

Technical onboarding

Create creator and account records, issue vault entries, assign least-privilege roles, import only approved media, configure tracking links, run one controlled publishing test and verify analytics before scaling.

43. Media ingest and rights metadata

Every media asset should enter through a controlled ingest point instead of being copied through random employee devices.

Content Operations Pipeline

Rights and approval precede distribution; performance data feeds the next creative decision

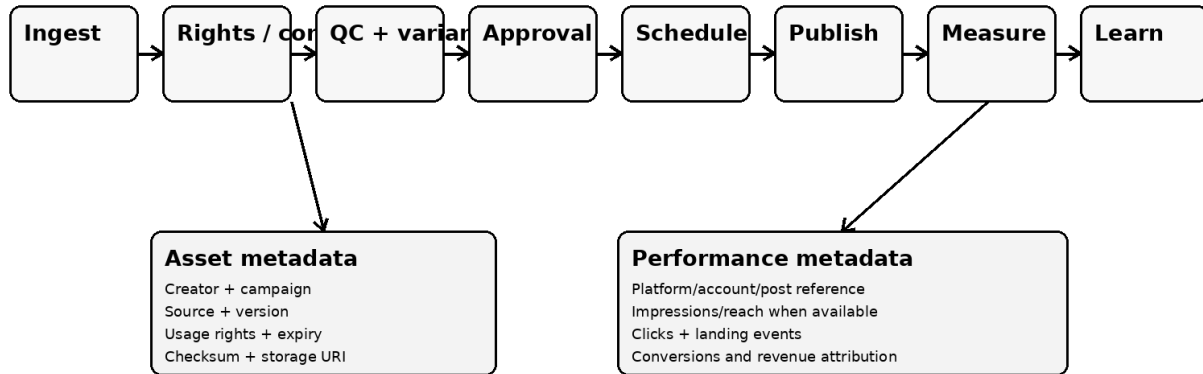


Figure 43-1. Rights-aware content pipeline.

Ingest

When an asset is received, calculate a checksum, store the original immutably, attach creator/campaign ownership, record source and rights scope, and create a QC status. The ingestion system should not silently strip or rewrite evidence relevant to rights and provenance.

If the agency performs edits, retain an edit lineage so the approved published variant can be traced to the source.

- Asset UUID.
- Checksum.
- Creator/campaign.
- Source.
- Rights/consent reference.
- Import timestamp.
- Classification.
- Retention/expiry.

Sensitive content

Adult material requires strict access control. Avoid unrestricted local synchronization. Use role-based access and watermarking for internal review only if it does not alter source evidence and the creator approves the workflow.

44. Content processing and variants

Content processing should optimize for platform-supported formats and campaign testing without deceptive manipulation.

Variant generation

Create explicit variants for aspect ratio, safe crops, subtitle/caption burn-in where authorized, cover frame, encoding bitrate and clip length. Store the transformation recipe and version ID so results are reproducible.

Use platform-published media specifications and test upload quality. Avoid “metadata spoofing” intended to make repeated content appear unrelated or to deceive integrity systems; that is different from legitimate transcoding or metadata minimization for privacy.

- 9:16 vertical variant.
- Platform-safe crop.
- Captioned variant.
- Thumbnail/cover.
- Source-preserving archival copy.

Quality control

Automated QC can detect resolution, duration, codec, audio presence, file corruption and duplicate checksum. Human QC should confirm brand, consent, visible private information and platform suitability.

45. Approval workflow

Approval is a gate, not a chat reaction. A task should prove which version was approved, by whom, for which account and time window.

Approval object

Approval should reference immutable asset version, creator/campaign, target platforms/accounts or a defined scope, approver identity, timestamp and expiration/revocation if applicable. A later edit creates a new version requiring new approval unless the policy explicitly allows that transformation.

High-risk or ambiguous content can require two-person review: creator/manager plus compliance/operations.

- Asset/version.
- Scope.
- Approver.
- Timestamp.
- Expiry.
- Notes/restrictions.

Revocation

If a creator withdraws authorization, the system should be able to identify queued tasks and previously published references associated with the revoked asset so operators can stop future use and execute contractual/platform removal procedures.

46. Publishing paths and official APIs

Publishing should use the most supportable path available: official API, approved scheduler/integration, or human-operated managed phone. The coordinator records all three the same way.

Official API path

TikTok's Content Posting API allows registered applications to post content on behalf of authorized creators after the app obtains the required product configuration and scope; unaudited clients have restrictions. [S22] This demonstrates the preferred automation pattern: explicit creator authorization, documented API scope and platform-controlled rate/feature constraints.

For other platforms, verify current first-party developer documentation because availability depends on account type, application review and changing product policy.

- OAuth/delegated authorization where supported.
- Rate limiting.
- Idempotency.
- Post/status ID capture.
- Audit of API actor and creator authorization.

Human device path

If an action is mobile-only or API-inaccessible, create an operator task with approved asset, caption guidance and target account. The operator performs the action in the native app and records verification. The coordinator should assist the human, not secretly impersonate the human.

47. TikTok operational notes

TikTok is particularly explicit about integrity and automation. Its current guidelines allow authentic multi-account use but prohibit spam, fake engagement, circumvention and automation used to register or operate accounts in bulk. [S23]

Compliant engineering posture

Use the Content Posting API when the desired workflow is supported and the application can satisfy authorization/audit requirements. Otherwise use trained operators on authorized creator accounts and keep posting cadence/content strategy inside current platform rules.

Build analytics around content quality and conversion rather than account multiplication. A smaller number of healthy, authorized accounts with measurable attribution is easier to govern than a disposable-account model.

- Do not automate bulk account creation.
- Do not buy/sell or misrepresent account identity.
- Do not manufacture views, likes, comments or follows.
- Do not build location-hiding or enforcement-circumvention mechanisms.

Operational telemetry

Record post ID, publishing path, asset version, timestamp, operator/API application, outcome and available reach/click data. If a post is rejected or account restricted, preserve the platform notice and route it to policy review rather than automatically creating replacements.

48. Reddit, X and Snapchat operational notes

Each platform has its own rules, but authenticity and spam controls are common themes.

Reddit

Reddit requires authentic participation and prohibits spam/content manipulation. [S24] Subreddit-specific rules can be stricter than site-wide rules. Build a subreddit rule register if Reddit is part of the creator strategy; do not assume an action allowed in one community is allowed in another.

Where API access is used, comply with current Reddit Data API terms and rate/access conditions. [S25]

- Record subreddit/community rules and last review date.
- Avoid repetitive cross-posting that violates community spam rules.
- Respect NSFW labeling and content restrictions.

X

X's authenticity rules recognize legitimate third-party management but prohibit coordinated inauthentic behavior, duplicate/substantially similar amplification and ban evasion. [S27] This supports a model where the agency manages authorized creator accounts without creating fake networks to manipulate reach.

Snapchat

Snapchat's deceptive-practices policy prohibits spam, artificially inflated engagement, follower-growth schemes and other deceptive behavior. [S26] Native content workflows should therefore remain human or platform-supported rather than driven by hidden engagement bots.

49. Instagram/Meta operational notes

Meta's products have formal business/developer tooling as well as anti-spam and authenticity rules. Exact publishing capabilities and account requirements change, so the deployment should treat Meta integrations as versioned connectors that are validated against current first-party documentation before release.

Source note

Meta policy/API pages can change frequently and were not used here as a step-by-step integration specification. Validate the exact current first-party endpoint, account type and review requirements at implementation time.

Architecture approach

Prefer supported Meta business/account access and approved publishing integrations where available. Maintain separate account ownership, page/profile/business relationships and authorization records in the control plane.

Avoid building the system around large-scale repetitive posting, synthetic engagement or replacement-account churn. Even if a device can technically perform an action, the business should first ask whether the action is permitted and sustainable.

- Verify current Instagram API publishing requirements.
- Use delegated business roles where available.
- Capture post/media IDs for reconciliation.
- Monitor access-token expiry and permission changes.

Native app tasks

Stories, certain editing features or emerging formats may reach the native app before APIs. Route these to a human task and maintain evidence of the approved asset and target account.

50. OnlyFans boundary and account responsibility

OnlyFans is the monetization endpoint in many OFM funnels, but the social phone farm and the OnlyFans account should be treated as separate systems. Agency assistance does not remove the creator's legal or contractual responsibility for the account. Publicly archived terms and UK parliamentary evidence have historically emphasized creator control/responsibility; verify current OnlyFans Terms directly before implementation. [S28][S29]

Verification warning

OnlyFans terms are especially important to verify at the time of deployment. Some accessible web sources are archived or secondary, so do not rely on this chapter as a current legal opinion.

Access

Do not assume that a social-media operations employee needs OnlyFans credentials. Separate traffic acquisition roles from subscriber/chat or account-administration roles. This reduces both privacy exposure and credential blast radius.

If third-party account assistance is permitted under current terms and creator contract, document the authority, employee scope, authentication method and revocation process.

- Creator retains required ownership/control.
- Agency access explicitly authorized.
- Separate roles and least privilege.
- MFA/recovery ownership documented.
- Offboarding and session revocation tested.

Data separation

Keep public social analytics, creator media and subscriber/private customer data in separate access domains. Do not copy subscriber conversations into a growth dashboard unless there is a documented lawful/business need and appropriate protection.

51. Tracking links and attribution

Without attribution, a phone farm can produce activity without knowing what generates revenue. Use campaign-aware links and first-party landing analytics where lawful.

Link structure

Create a campaign/link ID that resolves to the intended destination. Store creator, platform, account, campaign, asset and optional post reference. The redirect service records only necessary event data and should respect privacy/consent requirements.

UTM parameters are useful for general analytics, but an internal opaque link ID can be easier to rotate or update without exposing internal naming.

- Link ID.
- Creator/campaign.
- Source platform/account.
- Asset/post reference.
- Created/expired dates.
- Destination.

Attribution levels

Deterministic attribution uses a directly observed click and conversion identifier. Probabilistic or modeled attribution estimates influence when deterministic linking is unavailable. Label modeled results clearly; do not present them as exact facts.

Use cohort comparisons and revenue per campaign rather than optimizing solely for impressions.

52. Employee roles and access model

Employees are part of the system. Define roles so a new hire does not automatically receive every creator credential and media library.

Example roles

Fleet technician manages devices/network but does not need creator social passwords. Content operator sees assigned approved assets and target accounts. Growth manager sees analytics and campaign configuration. Security/admin manages access and incidents. Finance sees revenue/cost summaries but not raw sensitive content unless required.

Role-based access should be supplemented by creator/account scope. A content operator may be authorized for Creator A but not Creator B.

- Fleet technician.
- Content operator.
- Growth manager.
- Creator manager.
- Security/admin.
- Finance/analytics.

Joiner/mover/leaver

Provision named accounts, train the employee, issue hardware/keys, review access when role changes, and revoke access immediately at departure. Rotate shared recovery secrets if any existed, and reconcile devices/keys physically returned.

Role	Devices	Creator media	Social credentials	Analytics
Fleet tech	Admin	No/minimal	No	Health only
Content operator	Assigned	Assigned only	Delegated/assigned	Task-level
Growth manager	View	Campaign	Usually no password	Broad
Security admin	Admin	As incident requires	Vault admin only	Audit
Finance	No	No	No	Revenue/cost

53. Daily/weekly operating cadence

A phone farm should be run as an operations function with handoffs, not as continuous improvisation.

Daily

Start with health: offline devices, failed backups, full storage, expired credentials, network alerts and platform notices. Then review the approved publishing queue. End the shift with unresolved incidents and blocked tasks explicitly handed over.

Avoid measuring employees purely by number of posts; that incentivizes spam and hides quality or conversion problems.

- Morning fleet health check.
- Approval/queue review.
- Execute tasks.
- Verify and reconcile post IDs.
- Review alerts/restrictions.
- Shift handoff.

Weekly

Report output, funnel metrics, account health, device/network reliability, policy incidents, top/bottom content, labor usage and next-week experiments. The report should say what is blocked and who owns resolution.

54. Quality control and audit

Quality control verifies content, account and task correctness; audit verifies who changed what.

QC sampling

Randomly sample completed tasks and compare the published post with the approved asset/caption/target. Verify links resolve correctly and attribution metadata is present. Sample failed tasks to ensure error classification is honest rather than simply retried until “success.”

Periodically inventory physical devices against the database and MDM/Android Enterprise records.

- Published asset matches approval.
- Correct creator/account.
- Correct tracking link.
- Post ID captured.
- No unauthorized local asset remains.
- Device assignment matches physical rack.

Audit log

Write immutable or append-only records for login, role changes, credential access, creator authorization changes, task approvals, high-impact device commands and deletions. Retain according to legal/business requirements, not indefinitely by default.

PART VII - SECURITY, COMPLIANCE AND RELIABILITY

The fleet contains devices, credentials and sensitive creator content. Security therefore has to be designed into networking, software and employee workflows.

55. Threat model

A threat model identifies assets, adversaries, attack paths and controls. The most realistic threats are often credential theft, malicious/curious insiders, lost devices, vulnerable phones, exposed admin panels, leaked media and configuration mistakes.

Assets

Creator accounts, social sessions, content, legal authorization records, subscriber/private data where applicable, analytics/revenue, device inventory, network credentials and administrative secrets.

Rank assets by impact and limit who can access them. A fleet technician does not need subscriber data; a growth analyst does not need MDM wipe rights.

- External credential phishing.
- Malware on admin workstation.
- Stolen/unreturned phone.
- Insider media exfiltration.
- Misconfigured public storage.
- Compromised VPN/proxy account.
- Unpatched mobile OS.

Controls

MFA, vaulting, MDM, encryption, segmentation, EDR on admin hosts, patching, least privilege, secure backups, audit logs and employee offboarding cover most high-probability paths better than exotic anti-fraud tricks.

56. Platform restriction and enforcement handling

When a platform restricts a post or account, treat it as a policy/operations incident, not an automatic trigger to create replacement identities.

Workflow

Preserve the notice, account/post ID, timestamp, asset, publishing path and recent changes. Determine whether it is content policy, spam/integrity, security login, copyright, age/consent or an unknown issue. Use the platform's appeal/remediation process where appropriate.

Pause similar queued tasks if the issue may repeat. A single restriction can reveal a broken campaign rule that would otherwise affect dozens of accounts.

- Capture evidence.
- Classify cause.
- Contain similar tasks.
- Appeal/remediate legitimately.
- Update campaign/policy rule.
- Document decision.

Prohibited response pattern

Do not make "ban evasion" a software feature. Automated creation of replacement accounts, identity/fingerprint spoofing, location falsification or rapid proxy/device churn can violate platform rules and create fraud/security risk. TikTok and X explicitly address circumvention/ban evasion in their authenticity rules. [S23][S27]

57. Incident response

Incident response should be predefined because the first minutes of a credential leak or overheating event are not the time to invent authority and evidence rules.

Incident Response Flow

Design for containment and evidence before an incident happens

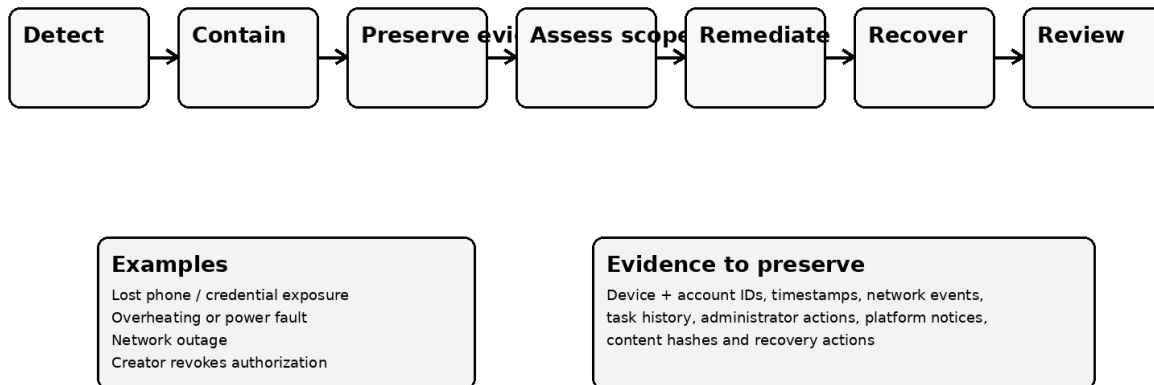


Figure 57-1. Incident-response sequence.

Seven stages

Detect -> contain -> preserve evidence -> assess scope -> remediate -> recover -> review. For a lost phone, containment may include MDM lock/wipe and secret revocation. For overheating, containment is physical power isolation and safe handling. For a content leak, containment includes access revocation and distribution/takedown actions consistent with law and platform processes.

Maintain an incident commander role so multiple employees do not issue conflicting changes.

- Incident ID and severity.
- Affected creator/account/device.
- Timeline.
- Actions/actors.
- Evidence.
- External notifications if required.
- Root cause and corrective action.

Testing

Run tabletop exercises for: lost device, compromised operator credential, database outage, storage exposure, creator revokes access, network failure during publishing, and rack thermal alarm.

58. Privacy and data minimization

A growth/fleet system should not become an uncontrolled surveillance database. Collect the minimum data needed to operate and secure the business.

Minimize

Device health does not require reading personal messages. Network health does not require storing full destination histories indefinitely. Analytics often need campaign/link IDs rather than raw IP addresses. Separate security logs from marketing data and apply different retention policies.

If employees operate creator accounts, document monitoring and privacy expectations in employment/contract policy and comply with applicable labor/privacy law.

- Purpose for each field.
- Retention period.
- Access role.
- Encryption requirement.
- Deletion/export process.

Data subject/creator exit

Be able to locate creator-specific records and assets for contractual deletion/return while preserving records the agency is legally required to retain. Avoid schemas that mingle multiple creators' sensitive content in untraceable folders.

59. Business continuity and creator exit

Continuity is not only infrastructure uptime; it includes a clean way for a creator to leave without the business or creator losing control of accounts.

Creator exit

Freeze new tasks, inventory pending posts, return/transfer authorized credentials or delegated roles according to contract/platform capabilities, revoke agency sessions, remove employee access, stop scheduled content, export agreed analytics/assets, and wipe creator material from reassigned devices.

Confirm recovery email/phone and MFA ownership before disabling agency access so the creator is not accidentally locked out.

- Stop queue.
- Reconcile scheduled/published content.
- Transfer/revoke access.
- Export/return agreed assets.
- Delete/wipe according to policy.
- Record completion.

Continuity

Document who can operate if the primary fleet admin is unavailable. Store network/MDM recovery procedures securely. At least two appropriately authorized people should be able to recover the system without sharing one administrator password.

PART VIII - SCALING AND IMPLEMENTATION

Scale in stages. A ten-device pilot should prove enrollment, physical design, network stability, queueing, attribution and incident recovery before the agency buys fifty more phones.

60. Scaling from 5 to 100+ devices

Scaling changes the bottleneck. At five devices, manual labeling and one AP may be adequate. At fifty, power zones, Wi-Fi capacity, management automation and staffing dominate. At one hundred, redundancy, change control and inventory discipline become mandatory.

Stage 1: 5-10

Goal: validate workflow. Use complete standard phones, one or two device models, managed Wi-Fi, a small controller and basic MDM/Android Enterprise. Measure actual power, heat, posting labor and conversion.

Do not optimize rack density yet. Optimize observability.

Stage 2: 25-50

Add segmented VLANs, multiple planned AP cells as needed, rack zones, UPS-backed network/control plane, automated enrollment, formal content approvals, task queue, metrics and scheduled maintenance. Separate backend/database from heavy media processing if necessary.

Stage 3: 100+

Use multiple physical racks/zones, standardized spares, automated inventory reconciliation, redundant network edge, tested disaster recovery, role-separated staff and canary change rings. Consider multiple sites only after one site is operationally mature.

Dimension	5-10	25-50	100+
Enrollment	Manual assisted	MDM/Enterprise standardized	Automated procurement/enrollment
Power	Single measured zone	Multiple protected zones	Rack/site capacity plan
Wi-Fi	1 planned AP may suffice	Capacity-driven multi-AP	RF design + monitoring
Controller	Single host	Separated roles as needed	HA services
Staffing	1-2 cross-functional	Defined roles	Shift + incident ownership

61. Cost model and BOM categories

Total cost is CAPEX plus recurring OPEX. A cheap phone can be expensive if it requires twice the labor or fails frequently.

CAPEX

Phones, rack/enclosure, chargers/PD supplies, cables, PDU, UPS, router/firewall, managed switch, APs, controller/workstations, environmental sensors, spare devices and installation materials.

Price bands vary strongly by country and used-device market. Obtain current local quotes rather than relying on a static web price in this manual.

- Devices.
- Rack/trays.
- Power distribution.
- Network.
- Controller/storage.
- Spares.
- Safety/environmental monitoring.

OPEX

Internet/cellular plans, MDM licenses, cloud hosting, object storage/egress, monitoring, domain/link infrastructure, employee labor, repairs/batteries, platform/developer costs where applicable, legal/compliance and backup storage.

Allocate shared infrastructure by campaign or creator using a transparent model. Labor is often the largest cost in a device-first workflow.

Cost class	Driver	Track as
Phone hardware	Device count/lifecycle	Monthly amortization
Power/network	Rack/site	Shared infrastructure
Software/MDM	Managed devices/users	Recurring license
Cloud/storage	GB, requests, compute	Usage
Connectivity	WAN/SIM plans	Monthly recurring
Labor	Tasks/hours	Direct operating cost

62. Pilot build - 10 device reference

The reference pilot is deliberately conservative. Its goal is to prove the system and gather measurements, not maximize density.

Hardware categories

Ten organization-owned phones from one or two supported model families; a ventilated non-compressing rack/tray; reputable multi-port charging or zoned chargers; labeled data cables for the subset needing USB admin; business firewall/router; managed PoE switch; one appropriately placed business AP; controller workstation; small UPS for network/controller; temperature sensor; spare phone and cables.

Do not purchase dedicated proxy infrastructure for the pilot unless there is a legitimate documented networking requirement. Direct business internet with proper segmentation is simpler to debug.

Software

MDM/Android Enterprise, inventory database, credential vault, basic coordinator/task board, object storage, tracking-link service, metrics/logging and a weekly report. TikTok or other official APIs can be integrated only after platform application authorization requirements are understood.

Pilot exit criteria

Every device can be enrolled/recovered, every account is authorized, every task is attributable, network/thermal behavior is stable, backup restore works, creator exit can be executed, and the business can calculate cost per successful campaign output.

Engineering checklist

- Run 72-hour full-load thermal/network test.
- Simulate one AP/router/controller restart.
- Wipe and re-enroll one device.
- Revoke one operator and verify access is gone.
- Publish/verify through every supported path.
- Restore database from backup.

63. 25-50 device expansion

Expansion is justified only after pilot measurements show stable operations and positive business value.

Physical expansion

Divide the rack into power/service zones, add airflow based on measured heat, add AP capacity based on measured airtime/utilization, and maintain at least 5-10% spare device capacity. Use port/slot maps and environmental alerting.

If a single controller still performs ADB, video transcoding, database and dashboard functions, split those roles before they become a single point of performance and failure.

Process expansion

Introduce formal change management: canary devices, scheduled OS/app updates, weekly access review for high-privilege roles, incident severity levels and capacity dashboards. Add a second trained administrator before the system becomes dependent on one person.

Software expansion

Move task execution to asynchronous queues, implement idempotency, add metrics/alerting, automate inventory reconciliation and make the creator/account authorization model explicit in every API request.

64. 30/60/90 day implementation roadmap

The roadmap assumes the agency starts with no mature device-management control plane. Adjust based on existing infrastructure.

Days 0-30 - prove the foundation

Inventory creators/accounts; define policy boundary; select 5-10 devices; deploy network segmentation; establish MDM/Android Enterprise; create rack/power/thermal pilot; deploy credential vault; implement creator/account/device/asset database; create manual task queue and tracking links; record baseline costs.

Deliverables: physical pilot, authorization register, network diagram, enrollment SOP, incident contact list, first weekly funnel report.

Days 31-60 - automate administration

Build coordinator API and companion/dashboard; add task state machine, approvals, object storage, health telemetry, metrics/alerts, canary updates and backup restore tests. Integrate one supported publishing API such as TikTok Content Posting API if business requirements and app authorization permit.

Deliverables: auditable task flow, fleet dashboard, monitoring, restore evidence, controlled API publishing path.

Days 61-90 - scale and govern

Expand to 25-50 only if KPIs justify it. Add power zones/AP capacity, formal roles, weekly access reviews, incident exercises, creator exit test, cost allocation, performance experiment process and quarterly platform-policy review.

Deliverables: scaled rack, runbooks, staff training, policy register, cost/conversion dashboard and go/no-go criteria for further scale.

65. Acceptance test plan

The installation is not complete because the phones turn on. Acceptance tests prove the physical, network, management, security and business workflows.

Physical/electrical

Measure AC load at idle and representative peak, verify no supply exceeds its rating, confirm rack temperatures in normal and degraded fan conditions, inspect cable strain and confirm emergency disconnect. Run a controlled power failure and verify UPS behavior for network/control systems.

Network/fleet

Verify DHCP/DNS/VLAN policy, Wi-Fi stability, tunnel/proxy behavior if configured, management reachability, enrollment, remote lock/wipe or equivalent supported actions, OS/app inventory and recovery after reboot.

Business/security

Create an approved asset, assign a task, publish via each supported path, capture post ID, verify tracked link, ingest analytics, produce weekly report, revoke an operator, remove a creator, restore a backup and execute one incident tabletop.

Engineering checklist

- All test evidence attached to acceptance record.
- Known exceptions have owner and due date.
- No production scaling before critical failures are closed.

APPENDIX A - Example IP/VLAN plan

Example only. Choose ranges that do not conflict with the organization's existing network or VPNs.

VLAN	Purpose	Example subnet	Allowed inbound	Primary egress
10	Management	10.20.10.0/24	Admin/VPN only	Direct corporate
20	Phone fleet A	10.20.20.0/24	Mgmt services only	Policy-defined
21	Phone fleet B	10.20.21.0/24	Mgmt services only	Policy-defined
30	Data/storage	10.20.30.0/24	API/worker only	Restricted
40	Quarantine	10.20.40.0/24	Mgmt only	Updates/support only
50	Guest	10.20.50.0/24	None	Direct isolated

Do not expose MDM connectors, ADB services, broker ports, database or storage administration directly to the public internet. Administrative access should traverse a secured management path.

APPENDIX B - Example firewall policy

Source	Destination	Service	Action	Reason
Mgmt VLAN	Fleet API	HTTPS	Allow	Admin/dashboard
Device VLAN	DNS resolvers	DNS/DoH/DoT as designed	Allow	Name resolution
Device VLAN	NTP/time	NTP/HTTPS time	Allow	Clock sync
Device VLAN	Internet	HTTPS + required platform traffic	Allow	Workload
Device VLAN	Mgmt VLAN	Any	Deny	Prevent lateral access
Device VLAN	Device VLAN	Any	Deny by default	Limit east-west
Quarantine	Internet	Updates/support subset	Allow limited	Recovery
Internet	Database/ADB/broker admin	Any	Deny	No direct exposure

The exact service list depends on Apple/Google management requirements, VPN/MDM vendor endpoints and platform applications. Use vendor-published network requirements and validate with staged tests.

APPENDIX C - Task JSON and API examples

These examples describe the internal coordinator. They do not contain commands for automating third-party social user interfaces.

```
POST /v1/tasks
{
  "creator_id": "cr_123",
  "account_id": "acct_tiktok_456",
  "asset_version_id": "assetv_789",
  "action": "PUBLISH_APPROVED_ASSET",
  "executor_type": "OFFICIAL_API_OR_HUMAN",
  "approval_ids": ["appr_001"],
  "campaign_id": "cmp_2026_08_a",
  "not_before": "2026-08-27T12:00:00Z",
  "deadline": "2026-08-27T18:00:00Z",
  "idempotency_key": "cmp...acct...asset..."
}
```

```
POST /v1/tasks/{id}/result
{
  "status": "VERIFIED",
  "executor_id": "operator_or_connector_id",
  "device_id": "dev_012",
  "platform_post_id": "platform-returned-id",
  "completed_at": "...",
  "evidence": {"type": "platform_post_reference"}
}
```

Recommended API safeguards

- OAuth/OIDC for employees; service credentials for connectors.
- Role and creator/account scope on every request.
- Idempotency keys for create/publish requests.
- Request ID and actor in every audit event.
- Schema validation and bounded text lengths.
- No plaintext platform passwords in request bodies.
- Rate limits for internal clients.

APPENDIX D - Example SQL data model

```
CREATE TABLE device (
  id UUID PRIMARY KEY, asset_tag TEXT UNIQUE NOT NULL,
  platform TEXT NOT NULL, model TEXT, serial_ref TEXT,
  management_state TEXT NOT NULL, rack_slot TEXT,
  last_heartbeat TIMESTAMPTZ, created_at TIMESTAMPTZ NOT NULL
);

CREATE TABLE platform_account (
  id UUID PRIMARY KEY, creator_id UUID NOT NULL, platform TEXT NOT NULL,
  handle TEXT, owner_type TEXT, authorization_ref TEXT NOT NULL,
  credential_ref TEXT, status TEXT NOT NULL
);

CREATE TABLE media_asset_version (
  id UUID PRIMARY KEY, creator_id UUID NOT NULL, source_asset_id UUID,
  sha256 TEXT NOT NULL, storage_uri TEXT NOT NULL,
  rights_ref TEXT NOT NULL, qc_status TEXT NOT NULL
);

CREATE TABLE task (
  id UUID PRIMARY KEY, creator_id UUID NOT NULL, account_id UUID NOT NULL,
  asset_version_id UUID, action TEXT NOT NULL, state TEXT NOT NULL,
  assigned_device_id UUID, assigned_operator_id UUID,
  idempotency_key TEXT UNIQUE, created_at TIMESTAMPTZ NOT NULL
);
```

Store sensitive serial/IMEI/recovery metadata with access controls or separate encrypted references as appropriate. The schema is illustrative and omits foreign keys/indexes for readability.

APPENDIX E - MQTT topic design

fleet/{device_id}/health	retained summary, device -> broker
fleet/{device_id}/events	transient events, device -> broker
fleet/{device_id}/commands	narrow approved internal commands
fleet/{device_id}/command-result	result acknowledgements
racks/{rack_id}/environment	temperature/power sensors
ops/incidents	security/operations event feed

Authorization model

- A device may publish only to its own health/events/result topics.
- A device may subscribe only to its own command topic.
- Rack sensors may publish only to their rack namespace.
- Operators use the API, not direct broker credentials.
- Broker admin interface is management-network only.
- TLS client authentication or strong per-device credentials.

APPENDIX F - Device commissioning checklist

Receiving

- ☐ Asset tag assigned
- ☐ Serial/model recorded
- ☐ Activation/carrier locks checked
- ☐ Battery/charging port inspected
- ☐ Factory reset performed

Enrollment

- ☐ Correct owner/enterprise
- ☐ MDM/Android Enterprise enrollment complete
- ☐ Supervision/fully-managed state verified
- ☐ Baseline policy applied
- ☐ Required apps installed

Network

- ☐ Correct SSID/VLAN
- ☐ DNS works
- ☐ VPN/proxy policy verified if applicable
- ☐ Management heartbeat received
- ☐ Public egress recorded for diagnostics

Security

- ☐ Passcode/lock policy
- ☐ Disk/device encryption active by platform default/management
- ☐ Debugging disabled unless required
- ☐ Credential vault assignment complete
- ☐ Lost/wipe procedure tested on cohort

Operations

- ☐ Rack slot assigned
- ☐ Cable/port mapped
- ☐ Tracking/companion app check-in
- ☐ Test task completed
- ☐ Status set to Ready

APPENDIX G - Weekly operations report template

Section	Required fields
Executive summary	What improved, what deteriorated, major risk/blocker, next decision
Fleet health	Online %, degraded/offline devices, thermal/power incidents, update status
Publishing	Tasks completed/failed, failure causes, approval latency, executor split
Account health	Restrictions/notices, appeals, account availability, policy changes
Funnel	Reach where available, clicks, conversion, revenue attribution confidence
Content	Top/bottom assets, experiments, QC/rejection rate
Security	Access changes, incidents, credential rotations, overdue reviews
Cost	Labor hours, connectivity/software, repair/amortization, estimated contribution
Next week	Priorities, experiments, maintenance, owner and deadline

APPENDIX H - Incident runbooks

Lost or stolen phone

1. Mark device Quarantined in coordinator.
2. Use MDM/Enterprise controls to lock/wipe as appropriate.
3. Revoke internal device token and any account sessions/credentials accessible from the device.
4. Preserve inventory/last-seen/network/task logs.
5. Notify creator/security owner if their account/content was exposed.
6. Replace from spare pool and reassign only after incident review.

Credential compromise

7. Disable/revoke compromised account or delegated session.
8. Rotate password/API token and MFA recovery material as required.
9. Search audit logs for access/use during exposure window.
10. Revoke employee/service sessions that inherited the secret.
11. Restore only after clean admin workstation and access path are verified.

Rack overheating

12. Stop new workload.
13. Inspect fan/airflow and ambient conditions.
14. Isolate abnormally hot/swollen devices; disconnect safely.
15. Reduce charging load or power down affected zone.
16. Do not return devices until temperatures and root cause are normal.

Platform restriction

17. Preserve notice and post/account IDs.
18. Pause similar queued tasks.
19. Classify policy/security/copyright/age-consent cause.
20. Use legitimate remediation/appeal path.
21. Update campaign rules; do not auto-create replacement identities.

Database outage

22. Pause writes/publishing that cannot be reconciled.
23. Verify storage/host health.
24. Fail over or restore according to runbook.
25. Reconcile queue vs database after recovery.
26. Document RPO/RTO achieved and any lost events.

APPENDIX I - Capacity formulas

Electrical planning

```
P_devices_avg = N_devices x measured_average_watts_per_device
P_total_avg   = P_devices_avg + P_network + P_controller + P_fans + P_storage
P_design      = P_total_expected x engineering_headroom_factor
```

Example only:

25 phones x 5 W measured average = 125 W device load

+ 120 W shared infrastructure = 245 W average

Design supplies/circuit/UPS from measured peak and required headroom, not this example.

Storage planning

```
Daily_media_GB = assets_per_day x average_asset_MB / 1024
Retention_GB   = Daily_media_GB x retention_days x version_multiplier
Backup_GB      = Retention_GB x backup_copy_factor
```

Add database, logs and temporary transcoding space separately.

Task throughput

```
Human_capacity_tasks_per_shift = operators x productive_minutes_per_shift / median_task_minutes
Required_concurrency_API       = peak_tasks_per_minute x p95_execution_minutes
```

Always apply platform/API rate limits and quality constraints before raw capacity targets.

Wi-Fi

Do not use a single “devices per AP” constant. Capacity depends on airtime, channel width, modulation/rate, interference, application traffic, AP radio design and environment. Measure channel utilization, retries and application latency under representative concurrent load.

APPENDIX J - Glossary

Term	Meaning
ADB	Android Debug Bridge; Google development/diagnostic bridge for authorized Android devices.
ACL	Access Control List; rule set permitting/denying network flows.
AP	Wireless access point.
APNs	Apple Push Notification service; used by MDM to signal devices that management work is available.
CGNAT	Carrier-Grade Network Address Translation; NAT performed in a carrier network at subscriber scale.
Control plane	Systems that manage policy, tasks, identity, inventory and administration.
Data plane	Actual media/application/network traffic produced by workload execution.
Dedicated device	Android Enterprise fully managed device locked to one or a small set of work purposes/apps.
DoH	DNS over HTTPS.
DoT	DNS over TLS.
Egress	Outbound path/address through which traffic leaves a network.
Idempotency	Property that allows the same logical request to be retried without creating duplicate effects.
MDM	Mobile Device Management.
MQTT	Lightweight publish/subscribe messaging protocol.
NAT	Network Address Translation.
PDU	Power Distribution Unit.
Proxy	Intermediary that makes/relays application connections on behalf of a client.
RBAC	Role-Based Access Control.
RPO	Recovery Point Objective; maximum tolerable data loss window.
RTO	Recovery Time Objective; target service restoration time.
Supervision	Apple organization-owned device management state providing additional controls.
TUN	Virtual network interface commonly used by VPN software.
VLAN	Virtual LAN; logical Layer-2 network segmentation.
VPN	Virtual Private Network; encrypted/logical tunnel to a network gateway.

APPENDIX K - Research references

Research cut: 26 August 2026. First-party sources are preferred. Secondary/archived sources are labeled. Platform policies can change; verify again before implementation.

ID	Source	URL / note	Quality
S1	Google Android Management API - Enroll and provision a device	https://developers.google.com/android/management/provision-device	First-party
S2	Google Android Management API - Policies reference	https://developers.google.com/android/management/reference/rest/v1/enterprises.policies	First-party
S3	Android Developers - Android Debug Bridge (adb)	https://developer.android.com/tools/adb	First-party
S4	Android Developers - VPN / VpnService	https://developer.android.com/develop/connectivity/vpn	First-party
S5	Apple Platform Deployment	https://support.apple.com/guide/deployment/welcome/web	First-party
S6	Apple Developer - Device Management	https://developer.apple.com/documentation/devicemanagement/	First-party
S7	Apple Developer - MDM commands and queries	https://developer.apple.com/documentation/devicemanagement/commands-and-queries	First-party
S8	Apple Platform Deployment - supervision	https://support.apple.com/guide/deployment/dep1d89f0bff/web	First-party
S9	AWS Device Farm Developer Guide	https://docs.aws.amazon.com/devicefarm/latest/developerguide/welcome.html	First-party; comparison/testing reference
S10	USB-IF - USB Charger / USB Power Delivery	https://www.usb.org/usb-charger-pd	First-party standards body
S11	Apple - Keeping iPhone/iPad within acceptable operating temperatures	https://support.apple.com/en-gb/118431	First-party
S12	Apple - iPhone battery and charging	https://support.apple.com/en-gb/105105	First-party
S13	Cisco - Wireless High Client Density Design Guide	https://www.cisco.com/c/en/us/td/docs/wireless/controller/technotes/8-7/b_wireless_high_client_density_design_guide.html	First-party vendor design guide
S14	Cloudflare - Forward/reverse proxy explanation	https://www.cloudflare.com/learning/cdn/glossary/reverse-proxy/	Vendor educational reference
S15	Cloudflare - A primer on proxies	https://blog.cloudflare.com/a-primer-on-proxies/	Vendor technical reference
S16	Cloudflare - DNS security	https://www.cloudflare.com/learning/dns/dns-security/	Vendor educational reference
S17	Cloudflare - DNS over TLS vs DNS over HTTPS	https://www.cloudflare.com/learning/dns/dns-over-tls/	Vendor educational reference
S18	APNIC - Carrier Grade NAT training material	https://training.apnic.net/wp-content/uploads/sites/2/2016/12/	Regional Internet registry training

		TIP602.pdf	
S19	WireGuard official site	https://www.wireguard.com/	First-party project
S20	MQTT official site	https://mqtt.org/	Standards/project reference
S21	Prometheus - Overview	https://prometheus.io/docs/introduction/overview/	First-party project
S22	TikTok for Developers - Content Posting API	https://developers.tiktok.com/doc/content-posting-api-get-started	First-party
S23	TikTok Community Guidelines - Integrity and Authenticity	https://www.tiktok.com/community-guidelines/en/integrity-authenticity/	First-party
S24	Reddit Rules	https://redditinc.com/policies/reddit-rules	First-party
S25	Reddit Data API Terms	https://redditinc.com/policies/data-api-terms	First-party
S26	Snap - Harmful False or Deceptive Practices	https://values.snap.com/policy/policy-community-guidelines/harmful-false-deceptive-practices	First-party
S27	X Help - Authenticity	https://help.x.com/en/rules-and-policies/authenticity	First-party
S28	UK Parliament written evidence quoting historical OnlyFans terms	https://committees.parliament.uk/writtenevidence/40780/default/	Official public record; historical
S29	Archived OnlyFans Terms snapshot	https://archive.ph/6mux7	Archive/secondary; verify current terms directly
S30	2026 secondary analysis of OnlyFans agency terms	https://whalefinders.com/blog/does-onlyfans-allow-agencies-tos-rules-2026	Secondary; unverified interpretation
S31	OSHA - Extension Cords and Power Strips Safety at Work (2026)	https://www.osha.gov/sites/default/files/publications/OSHA4495.pdf	US government safety guidance

Anecdotal industry material

During research, public OFM/phone-farm discussions were reviewed to understand terminology and claimed operating patterns. Self-described operators discuss old iPhone racks, multi-account posting, proxies, UI automation and other tactics. These reports are not independently verified, and some described tactics appear designed to bypass platform integrity controls. They are therefore not reproduced as implementation instructions in this guide. The architecture instead uses enterprise device-management concepts, approved APIs, human workflows and auditable networking.

Final engineering principle

Make the fleet observable, replaceable and accountable

A professional system should be able to answer, at any moment: Who authorized this account? Which approved asset was used? Which employee or connector executed the task? Which device/network path was involved? What result occurred? Can access be revoked immediately? Can the device be wiped and replaced? Can the business show whether the activity generated value? If those questions cannot be answered, adding more phones increases risk faster than capacity.

Source-use matrix

Topic	Primary sources
Android enrollment/policy/ADB/VPN	S1-S4
Apple enrollment/supervision/MDM	S5-S8
Device-farm comparison	S9
Power/thermal/safety	S10-S13, S31
Proxy/DNS/CGNAT/VPN	S14-S19
Messaging/monitoring	S20-S21
TikTok publishing/policy	S22-S23
Reddit/Snap/X rules	S24-S27
OnlyFans responsibility context	S28-S30 (historical/secondary caveat)